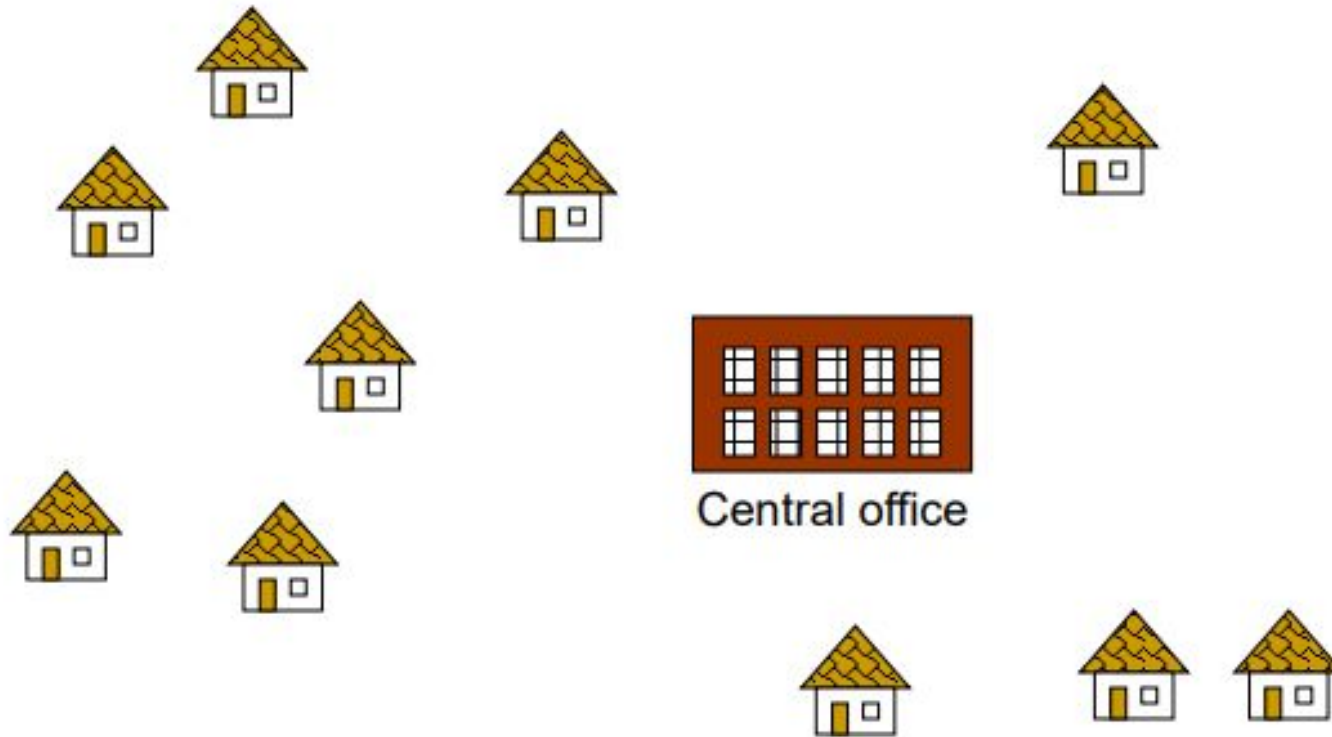


CSE-207

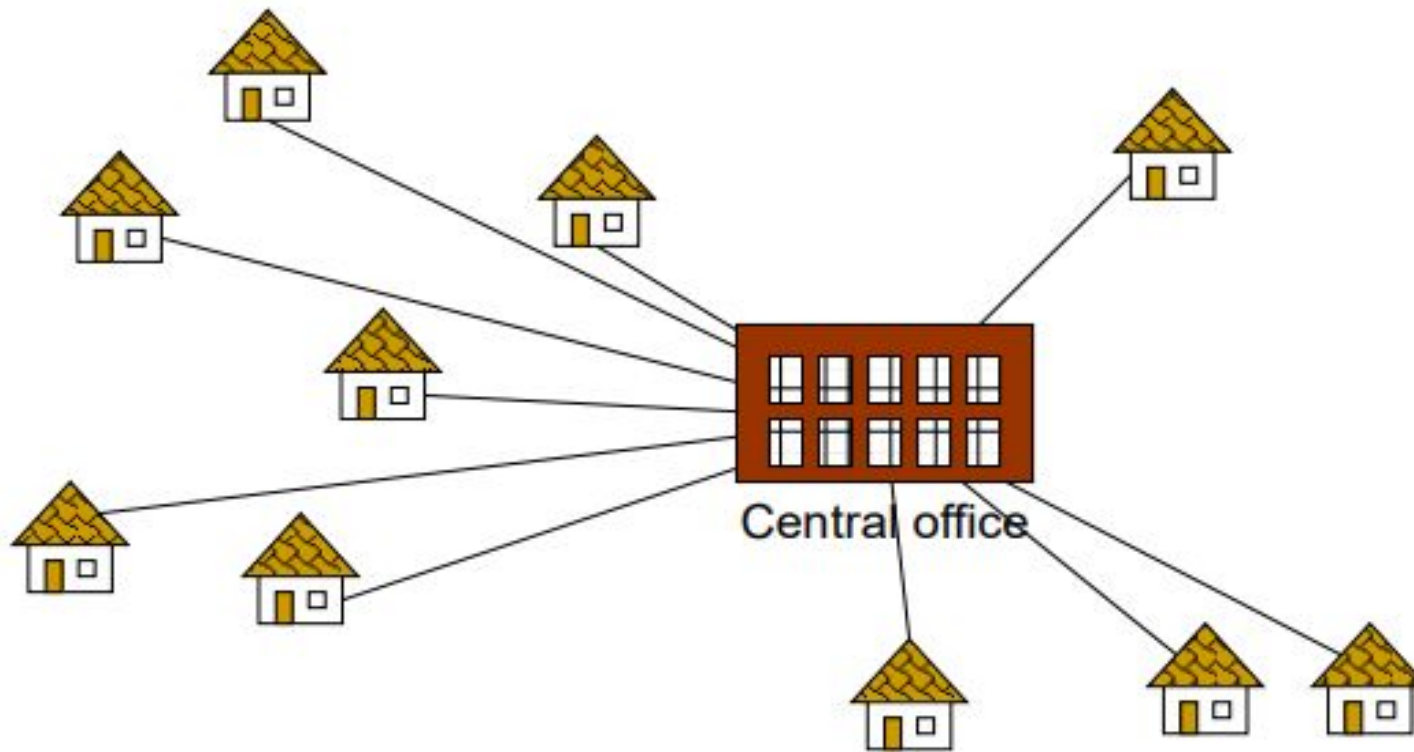
Lecture 5

Minimum spanning trees

Problem: Laying Telephone Wire

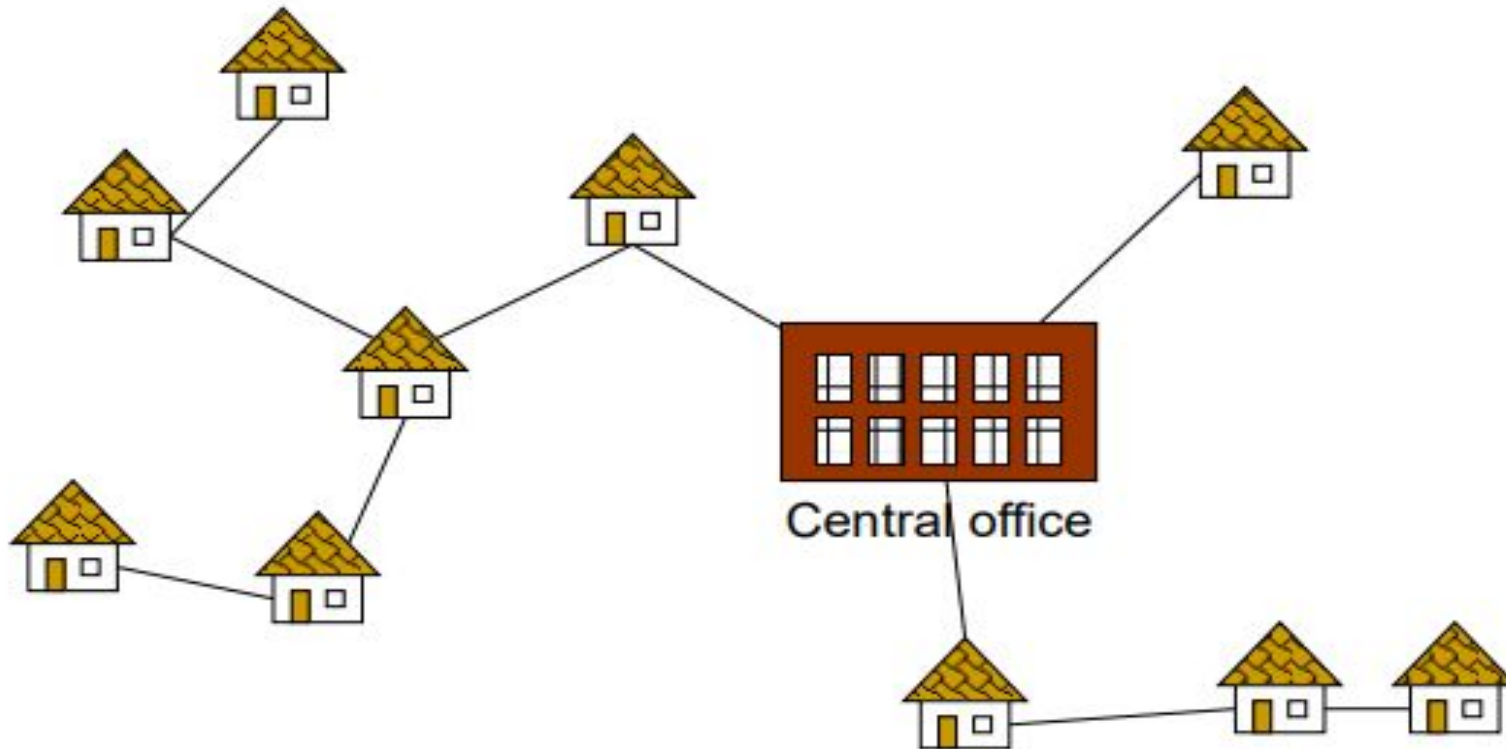


Wiring: Naïve Approach



Expensive!

Wiring: Better Approach

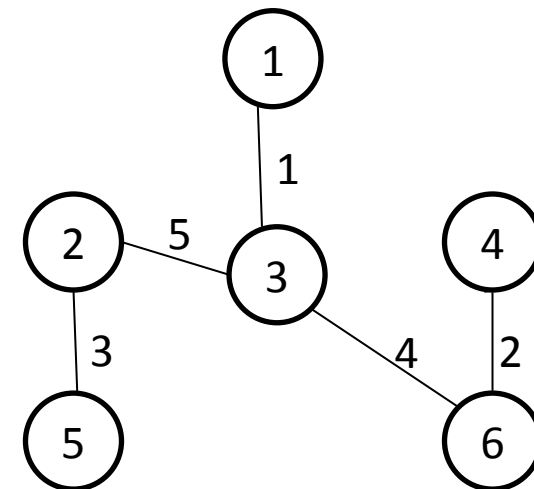
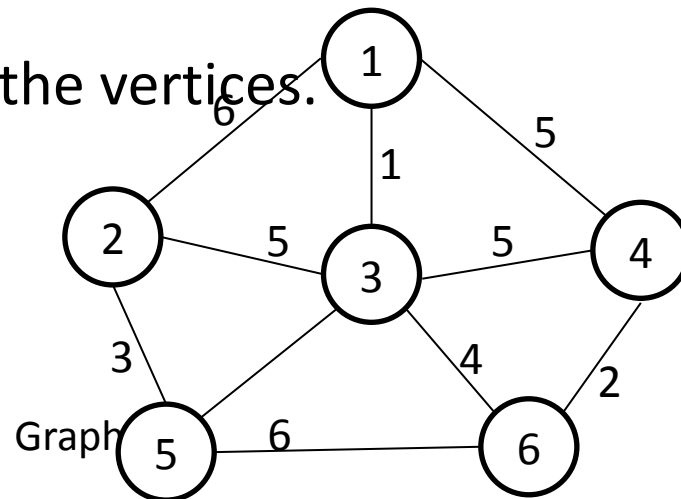


Spanning Tree

A minimum spanning tree (MTS) is a subgraph of an undirected graph such that the subgraph spans (include) all node is connected , is acyclic And has minimum total edge weight.

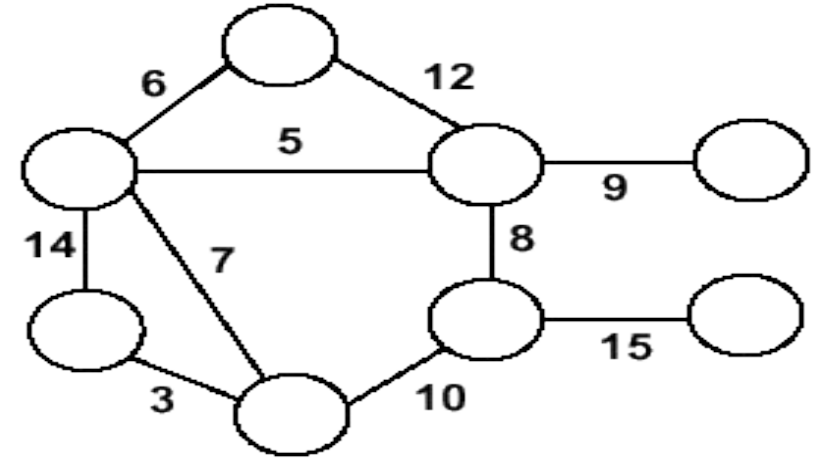
A spanning tree of G is a subgraph T that is:

- Connected.
- Acyclic.
- Includes all of the vertices.



Minimum Spanning Trees

- Undirected, connected graph $G = (V, E)$
- Weight function $W: E \rightarrow R$ (assigning cost or length or other values to edges)
 - Spanning tree: tree that connects all the vertices (above?)
 - Minimum spanning tree: tree that connects all the vertices and minimizes



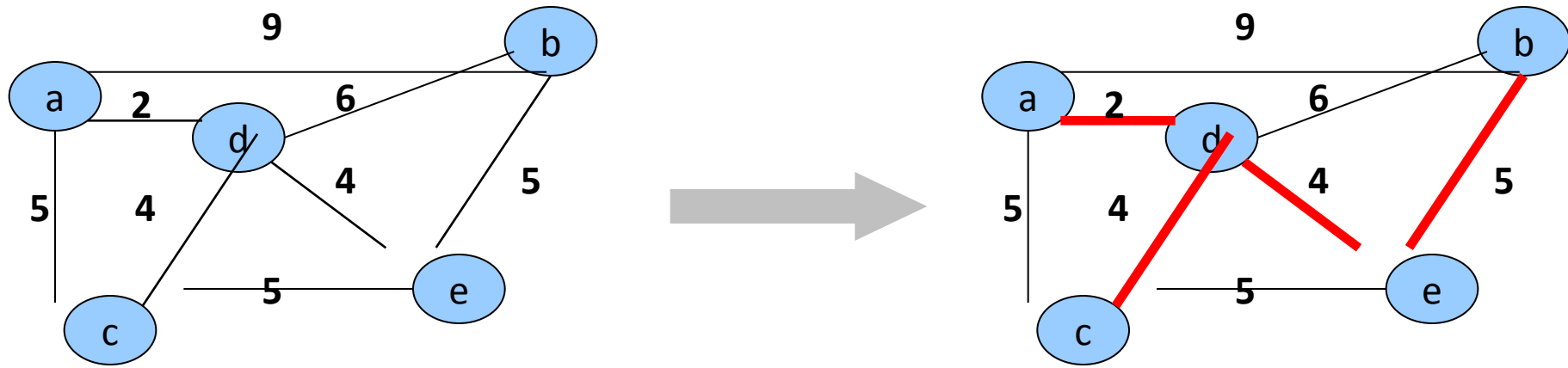
$$w(T) = \sum_{(u,v) \in T} w(u, v)$$

Minimum Spanning Tree (MST)

A **minimum spanning tree** is a subgraph of an undirected weighted graph G , such that

- it is a tree (i.e., it is acyclic)
- it covers all the vertices V
 - contains $|V| - 1$ edges
- the total cost associated with tree edges is the minimum among all possible spanning trees
- not necessarily unique

How Can We Generate a MST?



Greedy Choice

We will show two ways to build a minimum spanning tree.

- A MST can be grown from the current spanning tree by adding the nearest vertex and the edge connecting the nearest vertex to the MST.
(Prim's algorithm)
- A MST can be grown from a forest of spanning trees by adding the smallest edge connecting two spanning trees. (Kruskal's algorithm)

Notation

- **Tree-vertices:** in the tree constructed so far
- **Non-tree vertices:** rest of vertices

Prim's Selection rule

- Select the minimum weight edge between a tree-node and a non-tree node and add to the tree

Finding Spanning Trees

There are two basic algorithms for finding minimum-cost spanning trees, and both are greedy algorithms

- ❖ Kruskal's algorithm:
Created in 1956 by Joseph Kruskal
- ❖ Prim's algorithm
Created in 1957 by Robert C. Prim

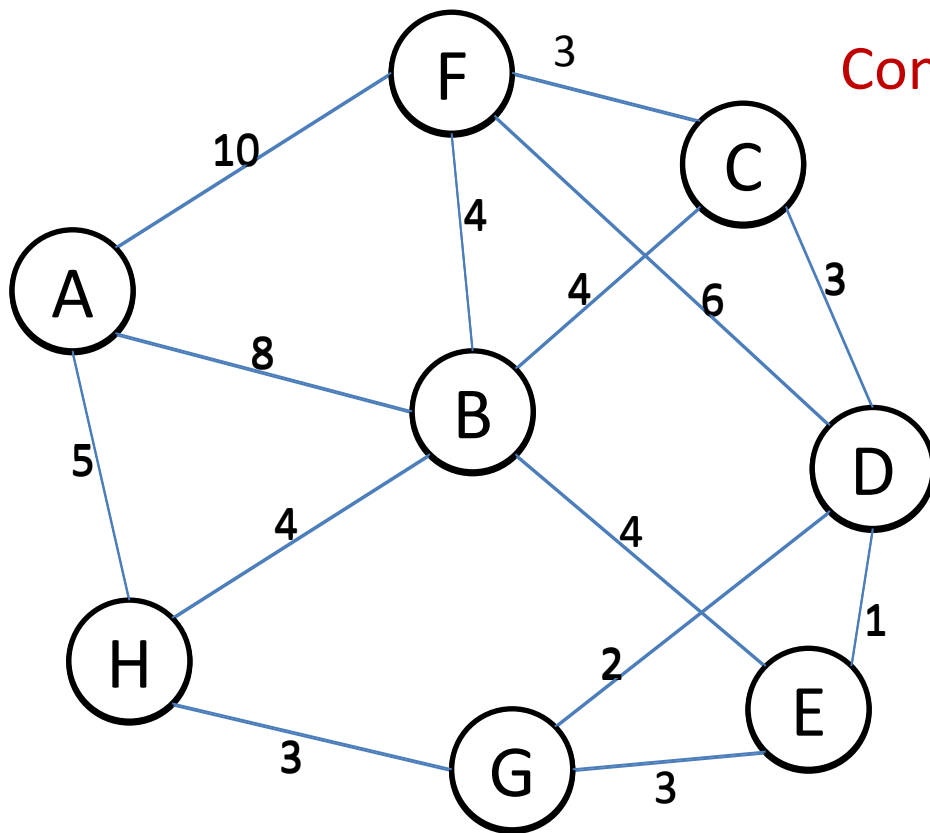
Algorithm Characteristics

- ❖ Both Prim's & Kruskal's algorithms work with undirected graph.
- ❖ Both work with weighted & unweighted graph.
- ❖ Both are Greedy algorithms that produce optimal solutions

Kruskal's Algorithm

- ✓ Select the shortest edge in a network.
- ✓ Select the next shortest edge which does not create a cycle.
- ✓ Repeat step 2 until all vertices have been connected.
- ✓ Work with edges, instead of node of a graph two step:
 - Sort edges by increasing edge weight
 - Select the first $|V|-1$ edges that do not generate a cycle

Kruskal's Algorithm

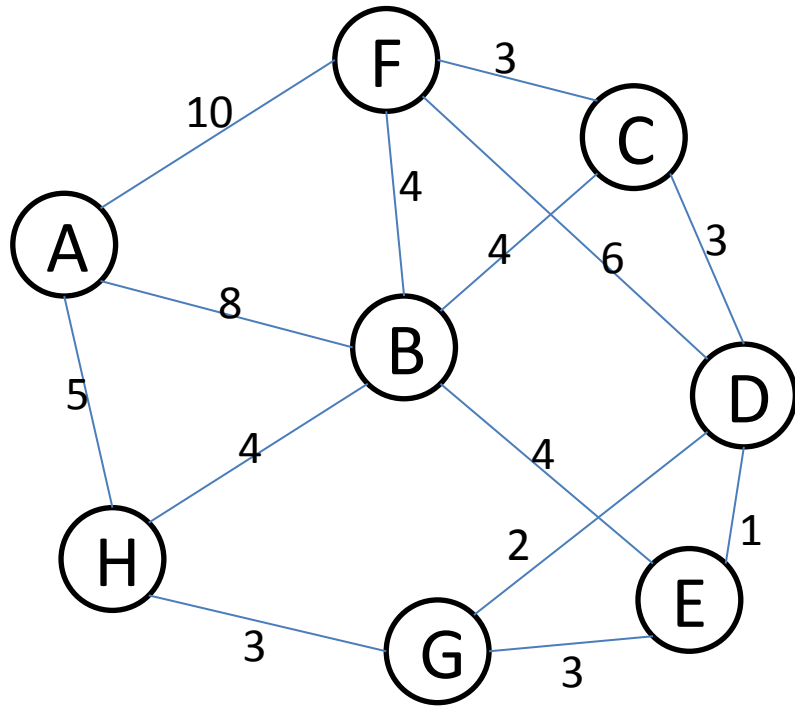


Consider undirected, Weight Graph

Kruskal's Algorithm

Sort the edges by increasing edge weight

– d_v : What is the minimal edge weight to v ?

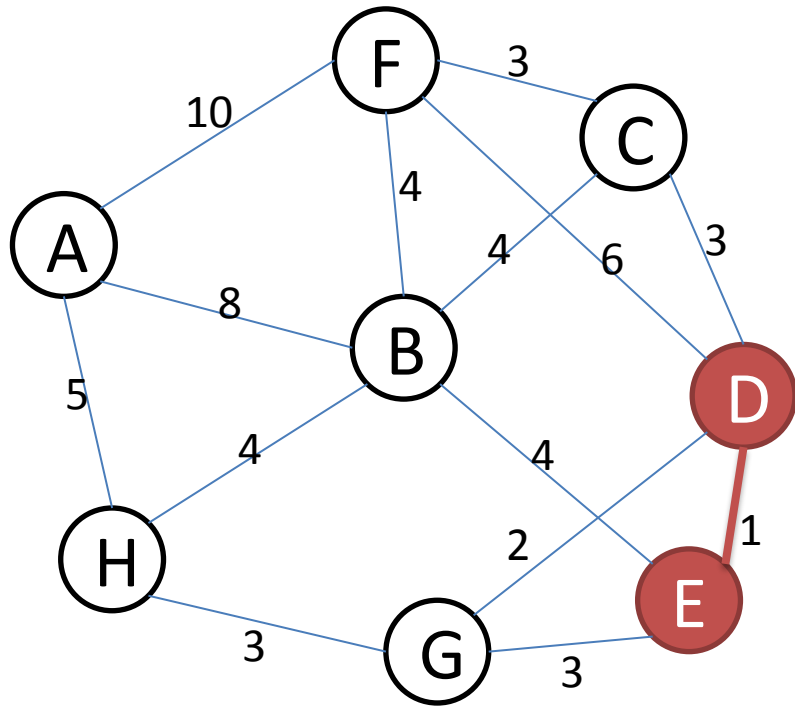


Edge	D_v	
(D,E)	1	
(D,G)	2	
(E,G)	3	
(C,D)	3	
(G,H)	3	
(C,F)	3	
(B,C)	4	

Edge	D_v	
(B,E)	4	
(B,F)	4	
(B,H)	4	
(A,H)	5	
(D,F)	6	
(A,B)	8	
(A,F)	10	

Kruskal's Algorithm

Select first $|V|-1$ edges which do not generate a cycle

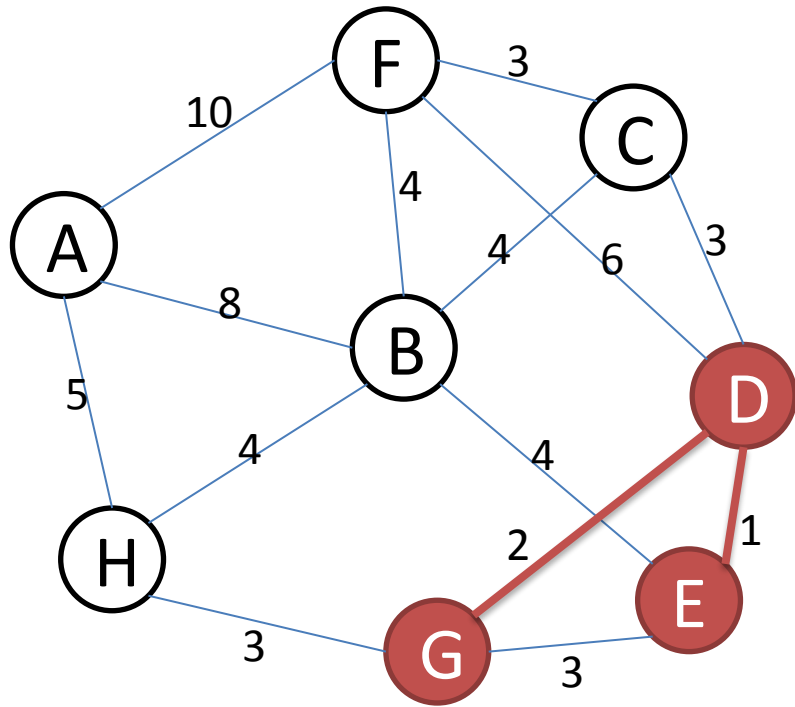


Edge	Dv	
(D,E)	1	✓
(D,G)	2	
(E,G)	3	
(C,D)	3	
(G,H)	3	
(C,F)	3	
(B,C)	4	

Edge	Dv	
(B,E)	4	
(B,F)	4	
(B,H)	4	
(A,H)	5	
(D,F)	6	
(A,B)	8	
(A,F)	10	

Kruskal's Algorithm

Select first $|V|-1$ edges which do not generate a cycle

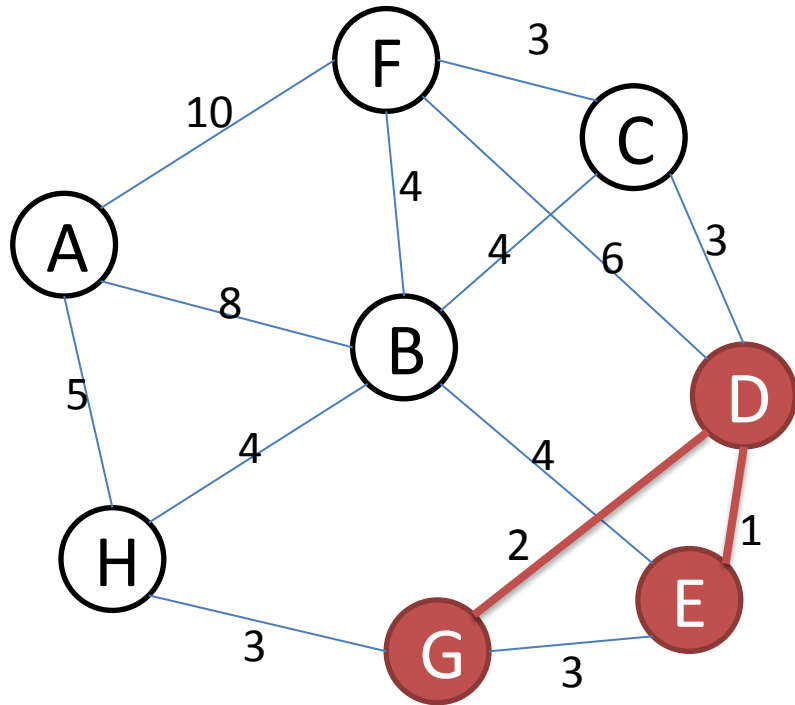


Edge	Dv	
(D,E)	1	✓
(D,G)	2	✓
(E,G)	3	
(C,D)	3	
(G,H)	3	
(C,F)	3	
(B,C)	4	

Edge	Dv	
(B,E)	4	
(B,F)	4	
(B,H)	4	
(A,H)	5	
(D,F)	6	
(A,B)	8	
(A,F)	10	

Kruskal's Algorithm

Select first $|V|-1$ edges which do not generate a cycle



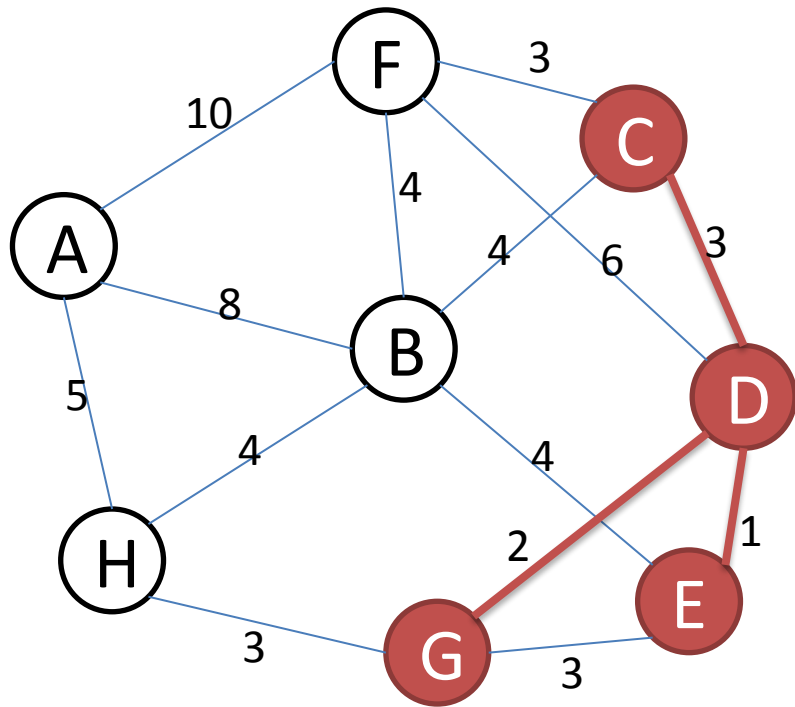
Edge	Dv	
(D,E)	1	✓
(D,G)	2	✓
(E,G)	3	X
(C,D)	3	
(G,H)	3	
(C,F)	3	
(B,C)	4	

Edge	Dv	
(B,E)	4	
(B,F)	4	
(B,H)	4	
(A,H)	5	
(D,F)	6	
(A,B)	8	
(A,F)	10	

... would create a cycle

Kruskal's Algorithm

Select first $|V|-1$ edges which do not generate a cycle

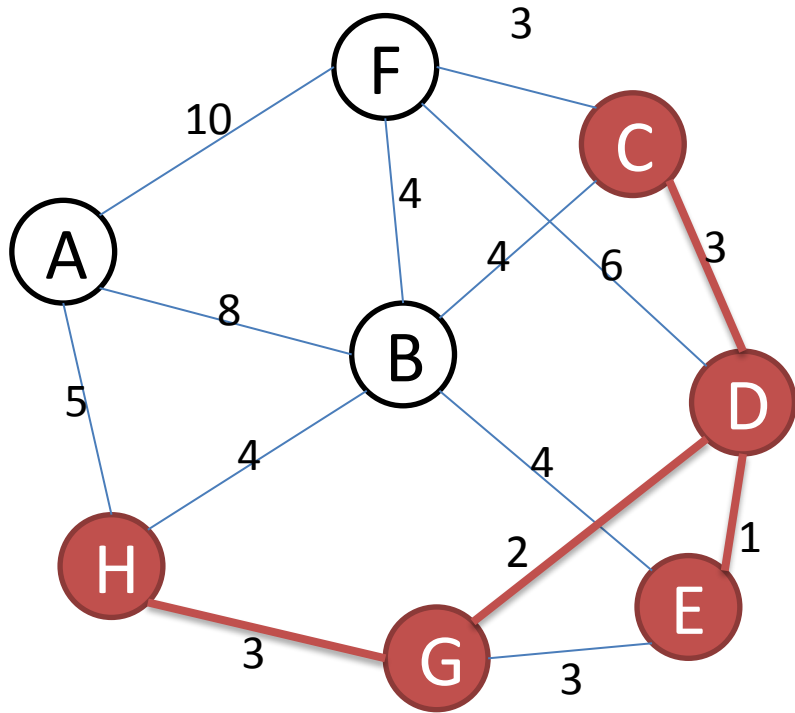


Edge	Dv	
(D,E)	1	✓
(D,G)	2	✓
(E,G)	3	X
(C,D)	3	✓
(G,H)	3	
(C,F)	3	
(B,C)	4	

Edge	Dv	
(B,E)	4	
(B,F)	4	
(B,H)	4	
(A,H)	5	
(D,F)	6	
(A,B)	8	
(A,F)	10	

Kruskal's Algorithm

Select first $|V|-1$ edges which do not generate a cycle

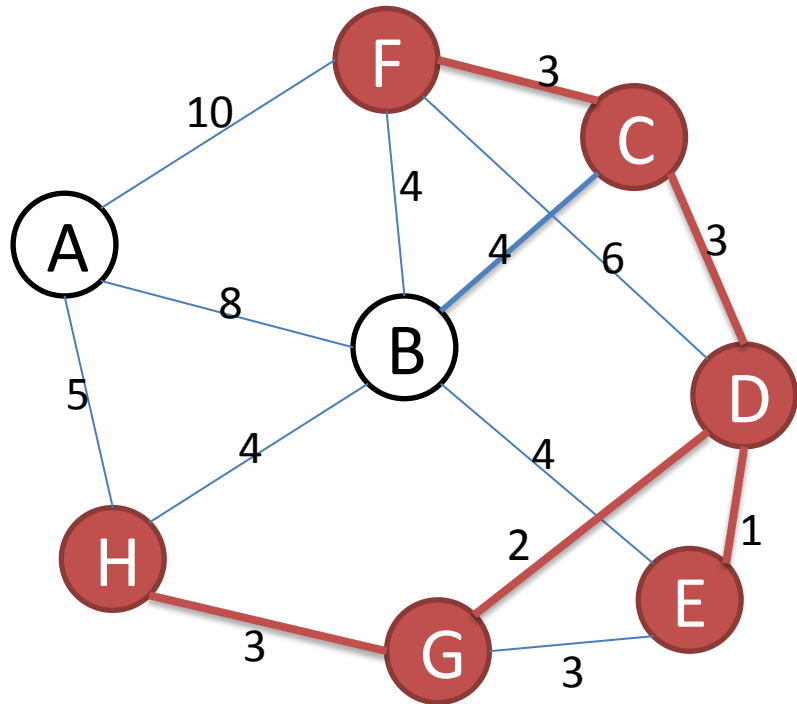


Edge	dv	
(D,E)	1	✓
(D,G)	2	✓
(E,G)	3	X
(C,D)	3	✓
(G,H)	3	✓
(C,F)	3	
(B,C)	4	

Edge	dv	
(B,E)	4	
(B,F)	4	
(B,H)	4	
(A,H)	5	
(D,F)	6	
(A,B)	8	
(A,F)	10	

Kruskal's Algorithm

Select first $|V|-1$ edges which do not generate a cycle

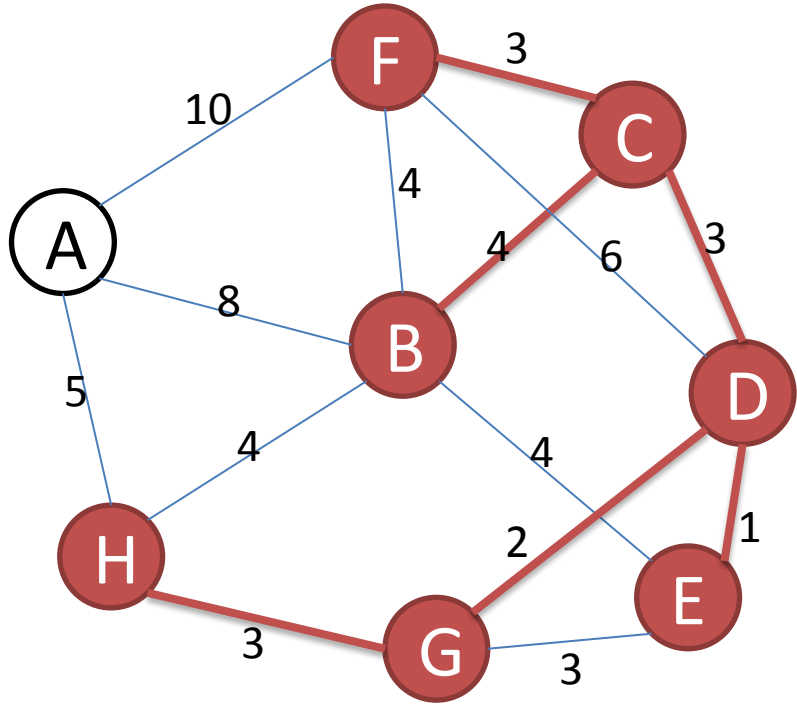


Edge	Dv	
(D,E)	1	✓
(D,G)	2	✓
(E,G)	3	X
(C,D)	3	✓
(G,H)	3	✓
(C,F)	3	✓
(B,C)	4	

Edge	Dv	
(B,E)	4	
(B,F)	4	
(B,H)	4	
(A,H)	5	
(D,F)	6	
(A,B)	8	
(A,F)	10	

Kruskal's Algorithm

Select first $|V|-1$ edges which do not generate a cycle

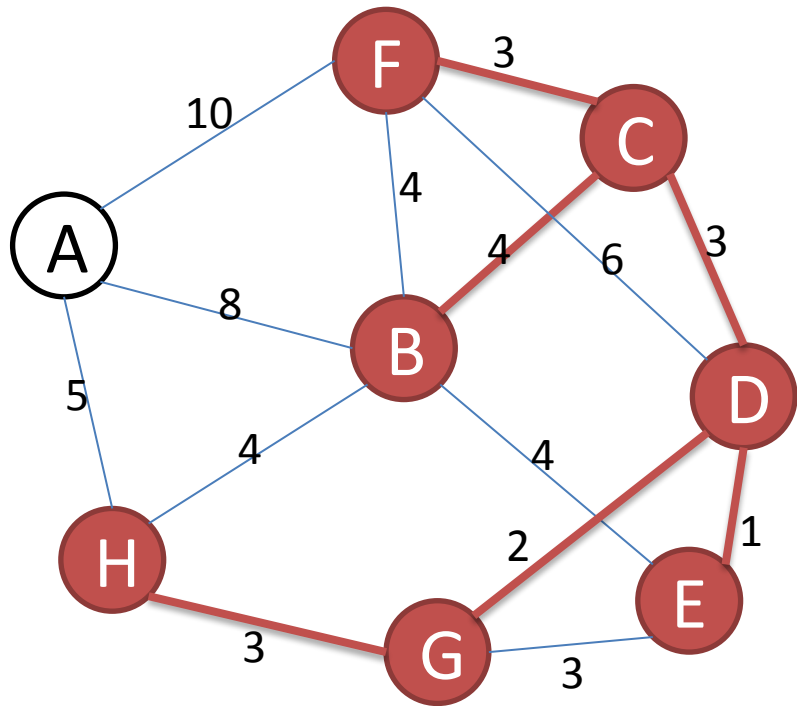


Edge	Dv	
(D,E)	1	✓
(D,G)	2	✓
(E,G)	3	X
(C,D)	3	✓
(G,H)	3	✓
(C,F)	3	✓
(B,C)	4	✓

Edge	Dv	
(B,E)	4	
(B,F)	4	
(B,H)	4	
(A,H)	5	
(D,F)	6	
(A,B)	8	
(A,F)	10	

Kruskal's Algorithm

Select first $|V|-1$ edges which do not generate a cycle

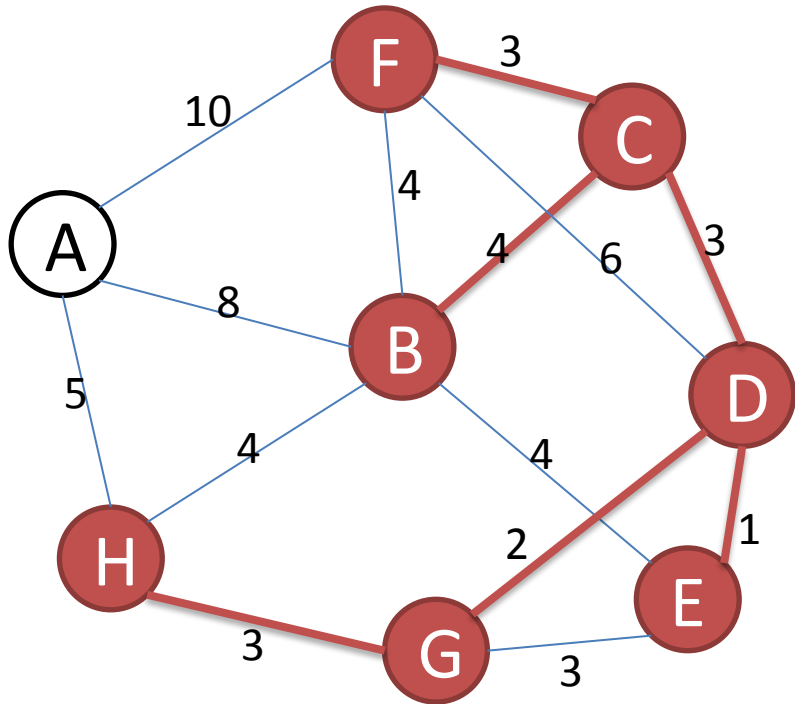


Edge	Dv	
(D,E)	1	✓
(D,G)	2	✓
(E,G)	3	X
(C,D)	3	✓
(G,H)	3	✓
(C,F)	3	✓
(B,C)	4	✓

Edge	Dv	
(B,E)	4	X
(B,F)	4	
(B,H)	4	
(A,H)	5	
(D,F)	6	
(A,B)	8	
(A,F)	10	

Kruskal's Algorithm

Select first $|V|-1$ edges which do not generate a cycle

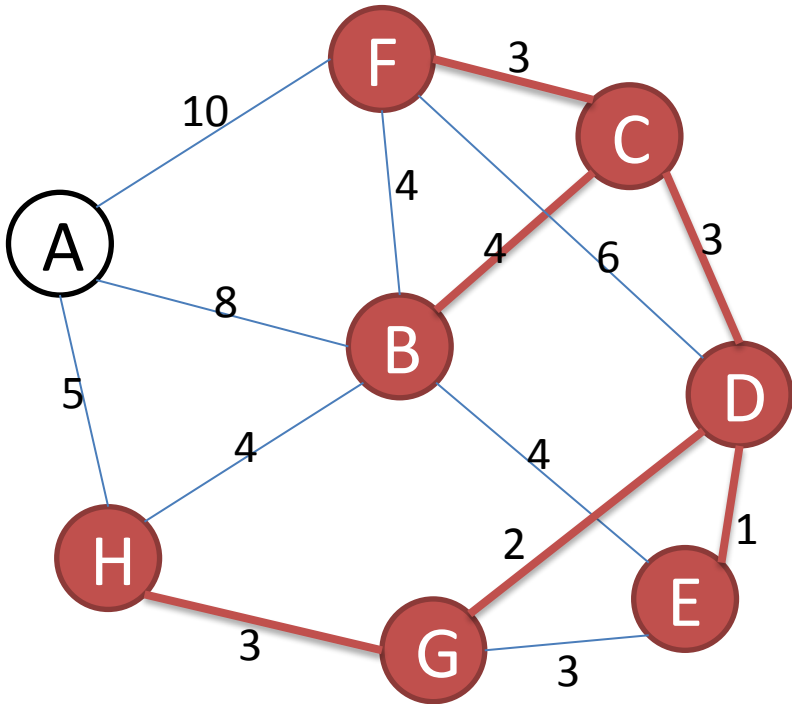


Edge	Dv	
(D,E)	1	✓
(D,G)	2	✓
(E,G)	3	X
(C,D)	3	✓
(G,H)	3	✓
(C,F)	3	✓
(B,C)	4	✓

Edge	Dv	
(B,E)	4	X
(B,F)	4	X
(B,H)	4	
(A,H)	5	
(D,F)	6	
(A,B)	8	
(A,F)	10	

Kruskal's Algorithm

Select first $|V|-1$ edges which do not generate a cycle

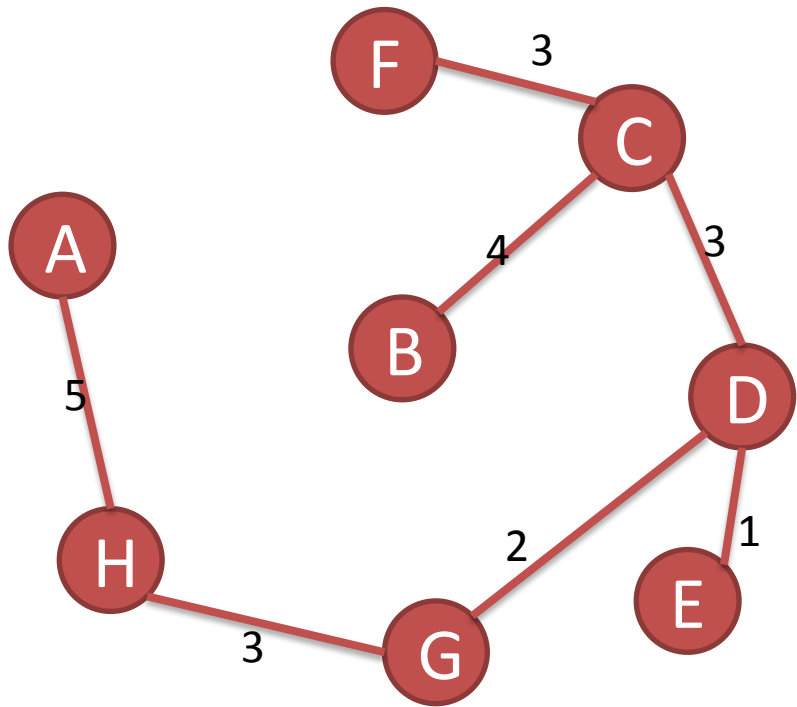


Edge	Dv	
(D,E)	1	✓
(D,G)	2	✓
(E,G)	3	X
(C,D)	3	✓
(G,H)	3	✓
(C,F)	3	✓
(B,C)	4	✓

Edge	Dv	
(B,E)	4	X
(B,F)	4	X
(B,H)	4	X
(A,H)	5	
(D,F)	6	
(A,B)	8	
(A,F)	10	

Kruskal's Algorithm

Select first $|V|-1$ edges which do not generate a cycle



Total Cost = 21

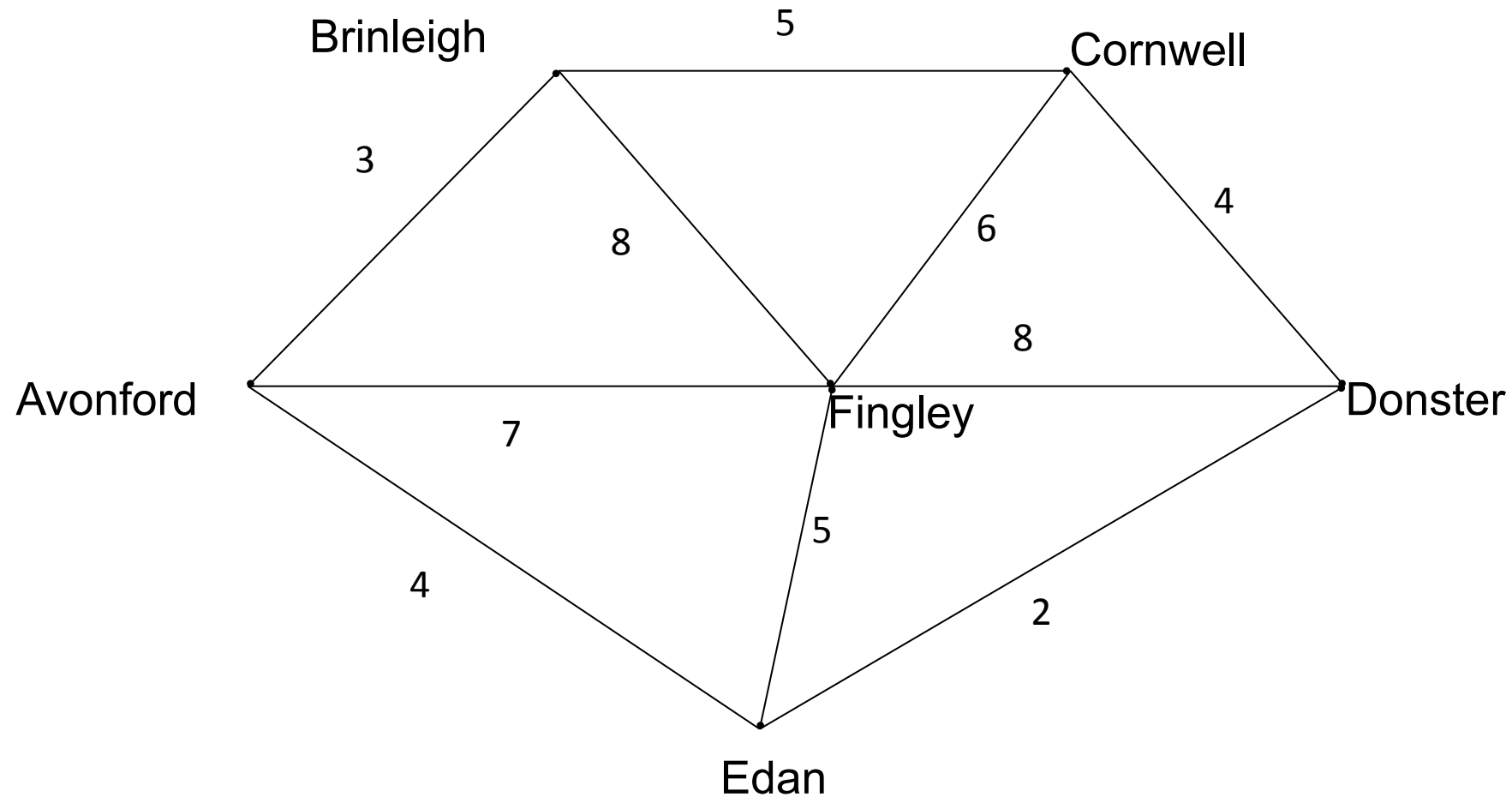
Edge	Dv	
(D,E)	1	✓
(D,G)	2	✓
(E,G)	3	X
(C,D)	3	✓
(G,H)	3	✓
(C,F)	3	✓
(B,C)	4	✓

Edge	Dv	
(B,E)	4	X
(B,F)	4	X
(B,H)	4	X
(A,H)	5	✓
(D,F)	6	
(A,B)	8	
(A,F)	10	

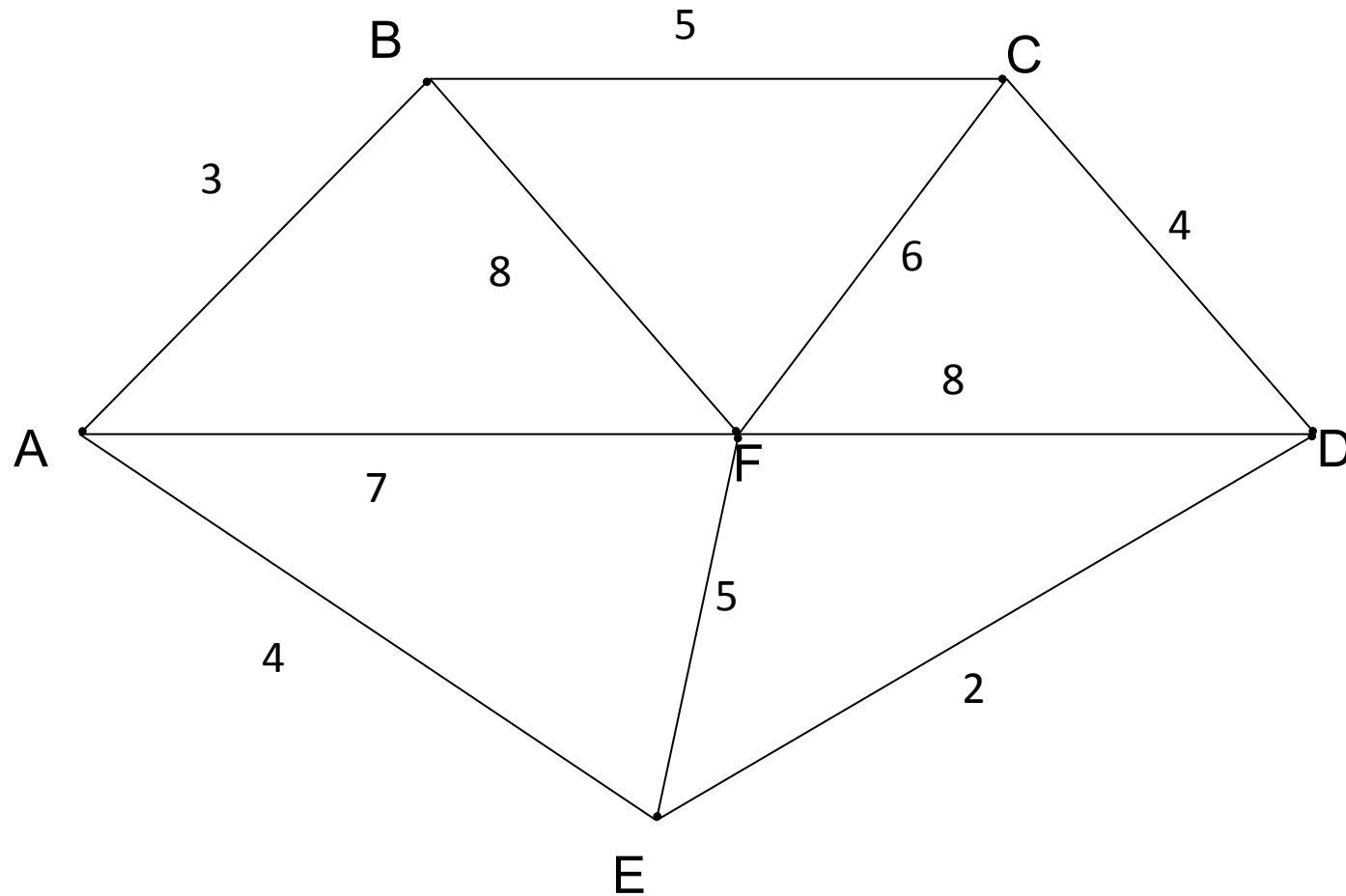
not
consider

Example

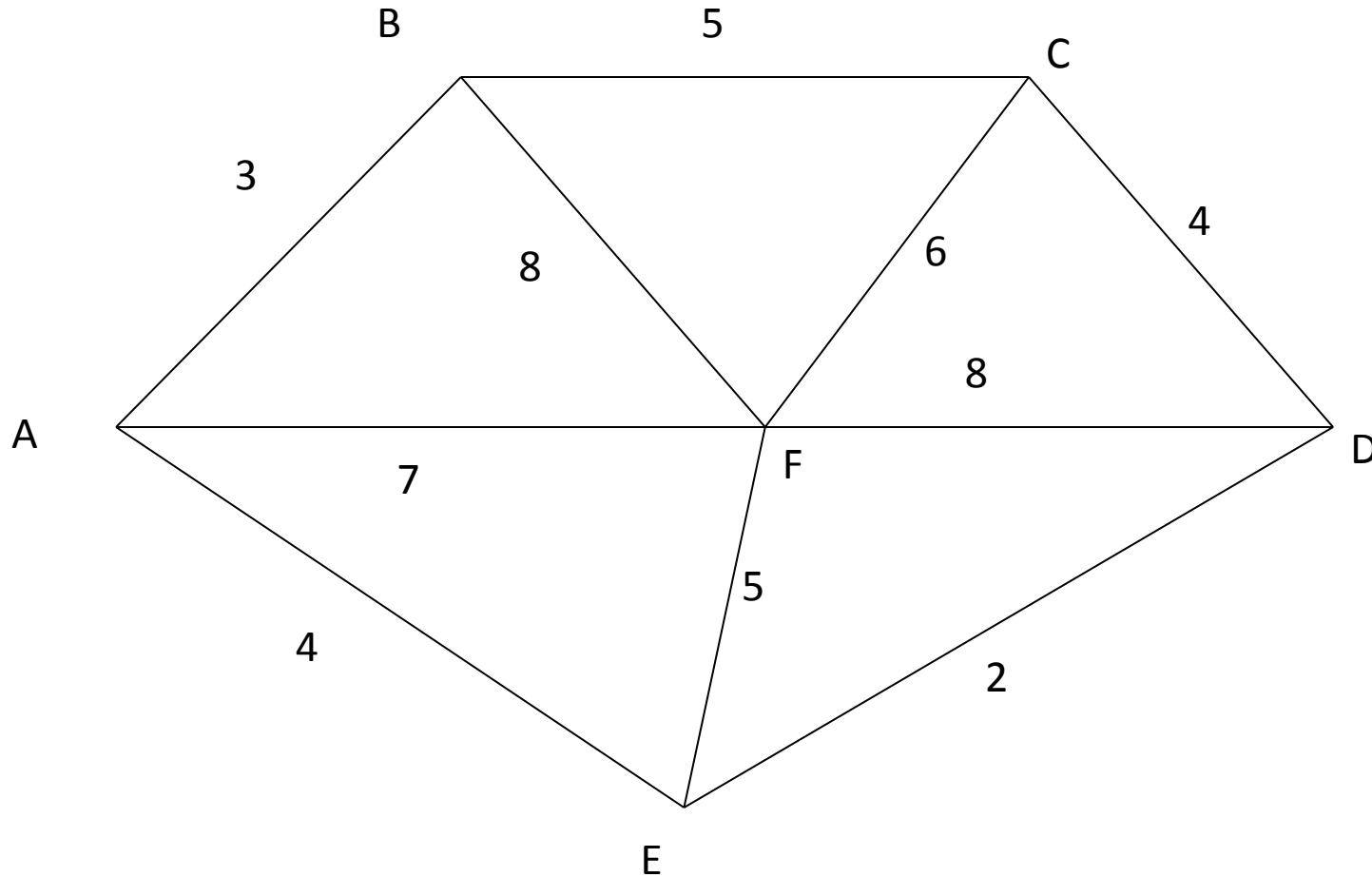
A cable company want to connect five villages to their network which currently extends to the market town of Avonford. What is the minimum length of cable needed?



We model the situation as a network, then the problem is to find the minimum connector for the network



Kruskal's Algorithm

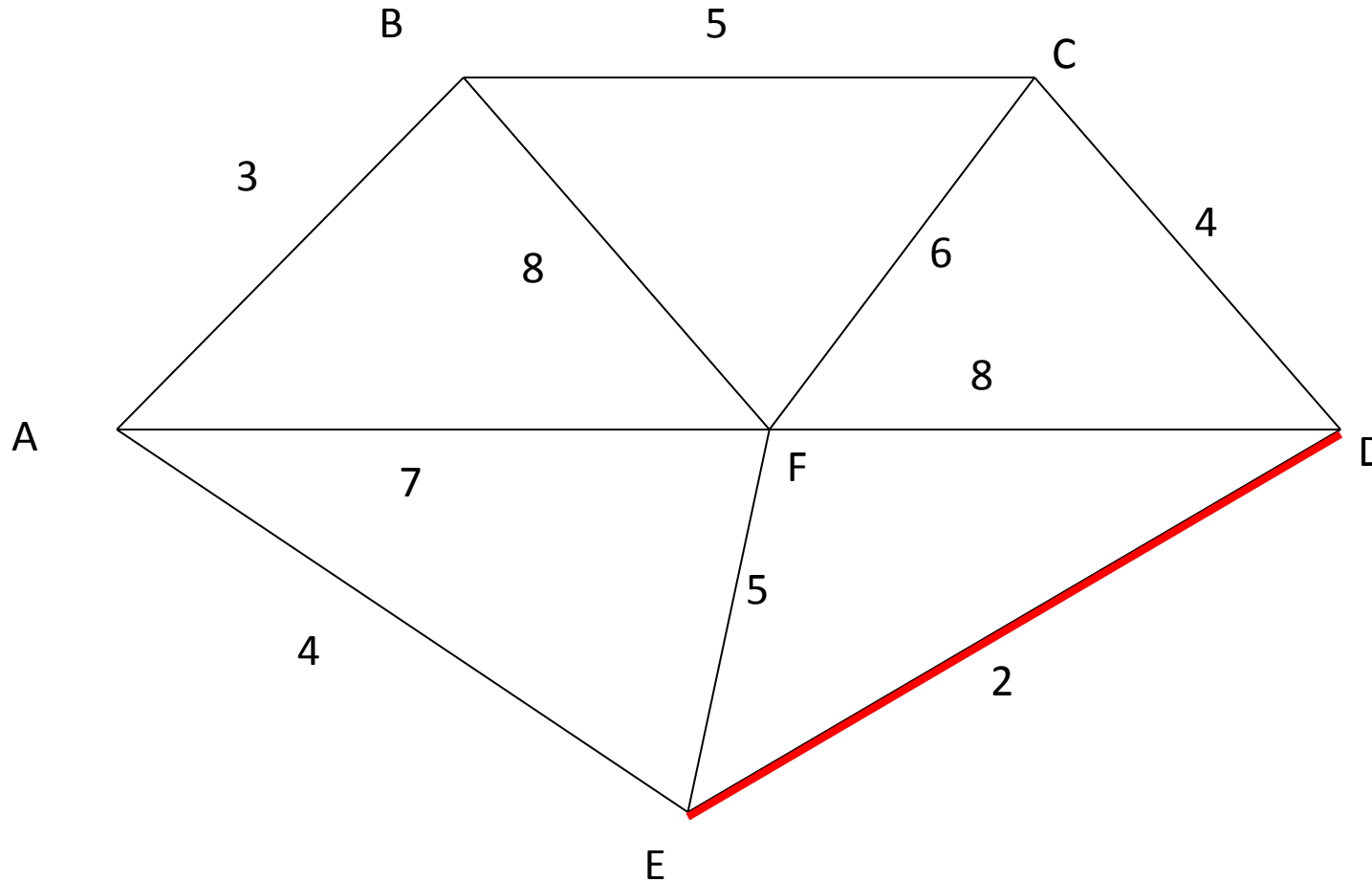


List the edges in order of size:

ED 2
AB 3
AE 4
CD 4
BC 5
EF 5
CF 6
AF 7
BF 8
DF 8

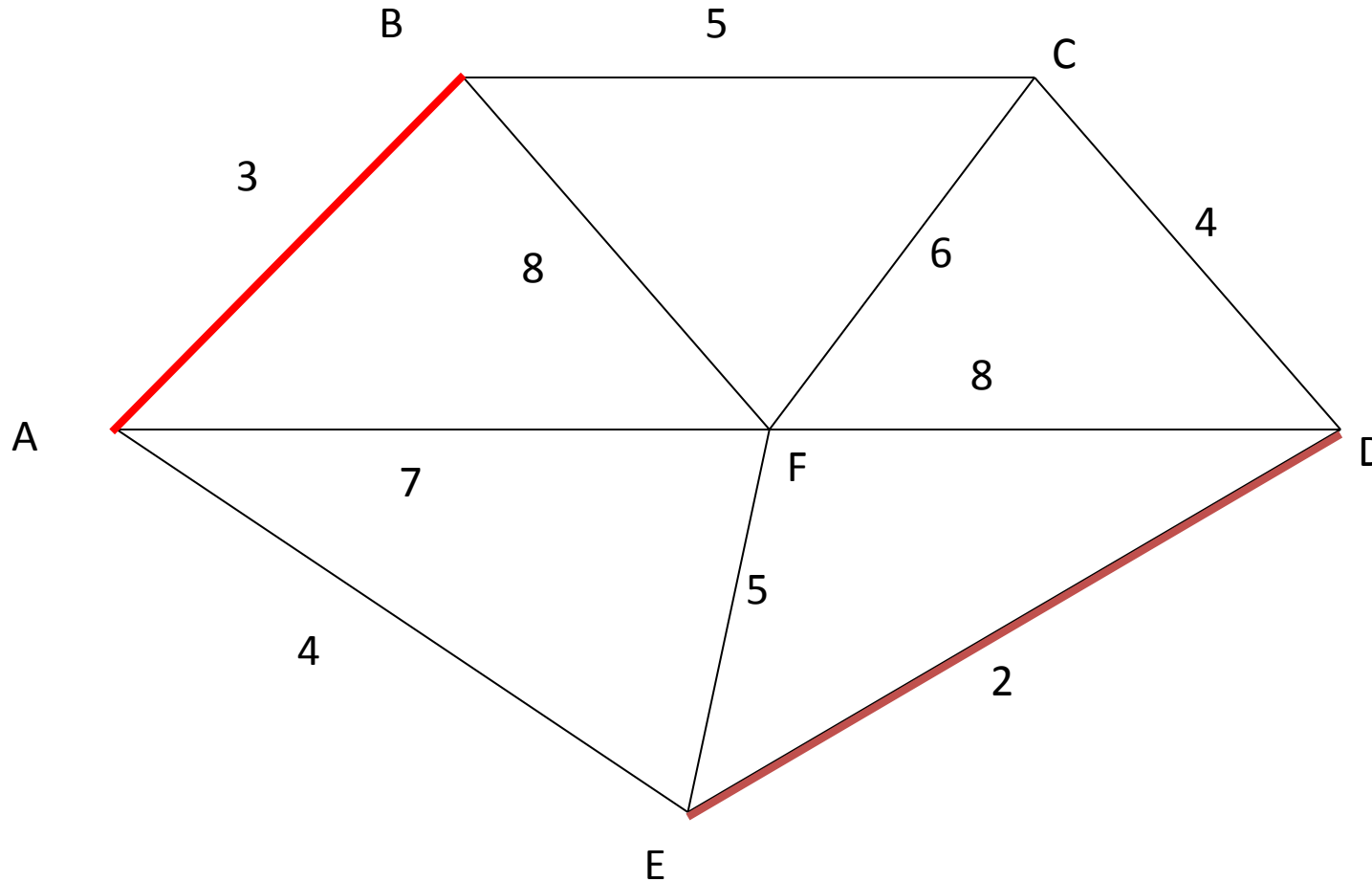
Kruskal's Algorithm

Select the shortest edge in the network



ED	2
AB	3
AE	4
CD	4
BC	5
EF	5
CF	6
AF	7
BF	8
DF	8

Kruskal's Algorithm

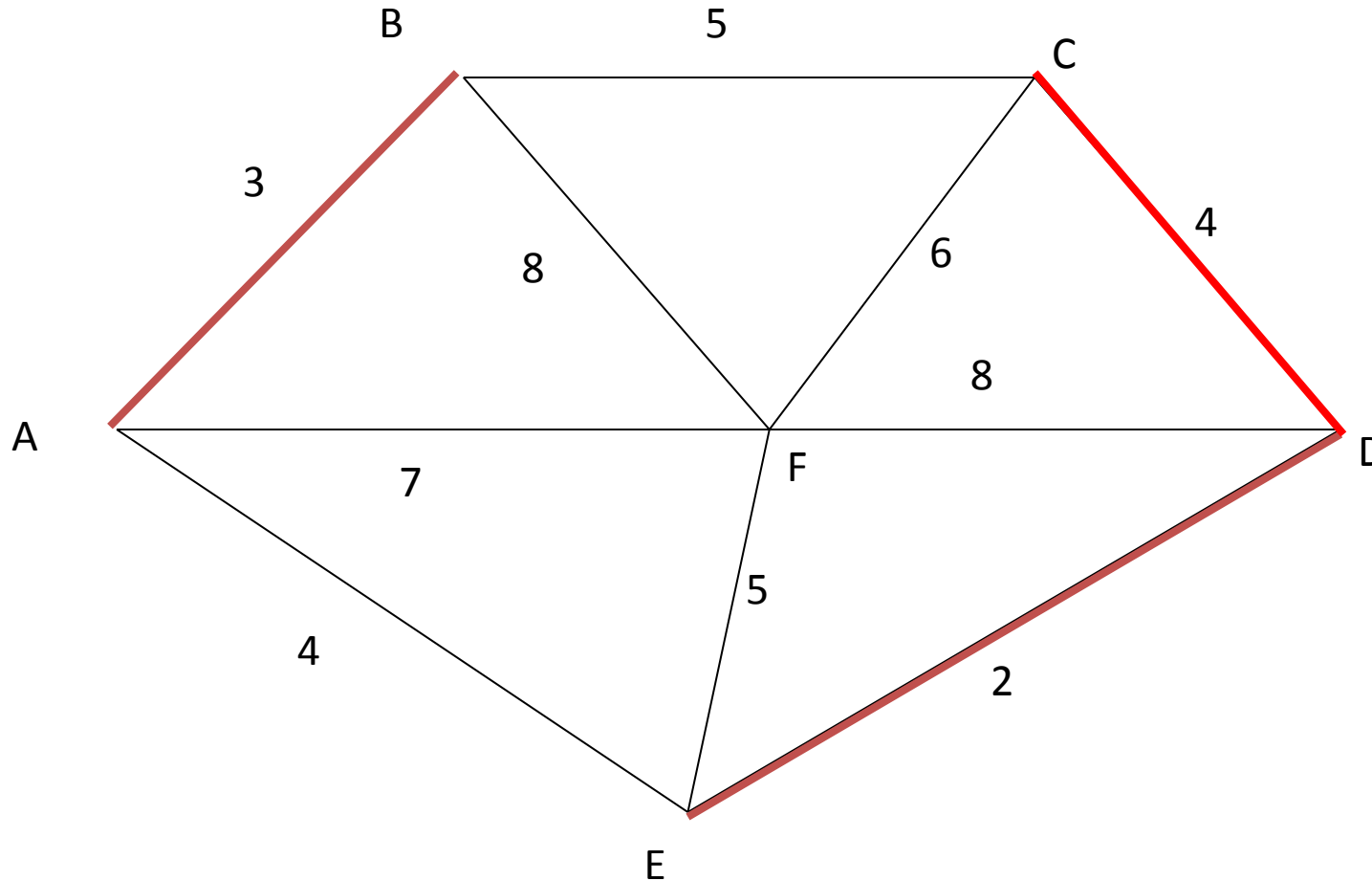


Select the next shortest edge which does not create a cycle

ED 2
AB 3

ED 2
AB 3
AE 4
CD 4
BC 5
EF 5
CF 6
AF 7
BF 8
DF 8

Kruskal's Algorithm



Select the next shortest edge which does not create a cycle

ED 2

AB 3

CD 4 (or AE 4)

ED 2

AB 3

AE 4

CD 4

BC 5

EF 5

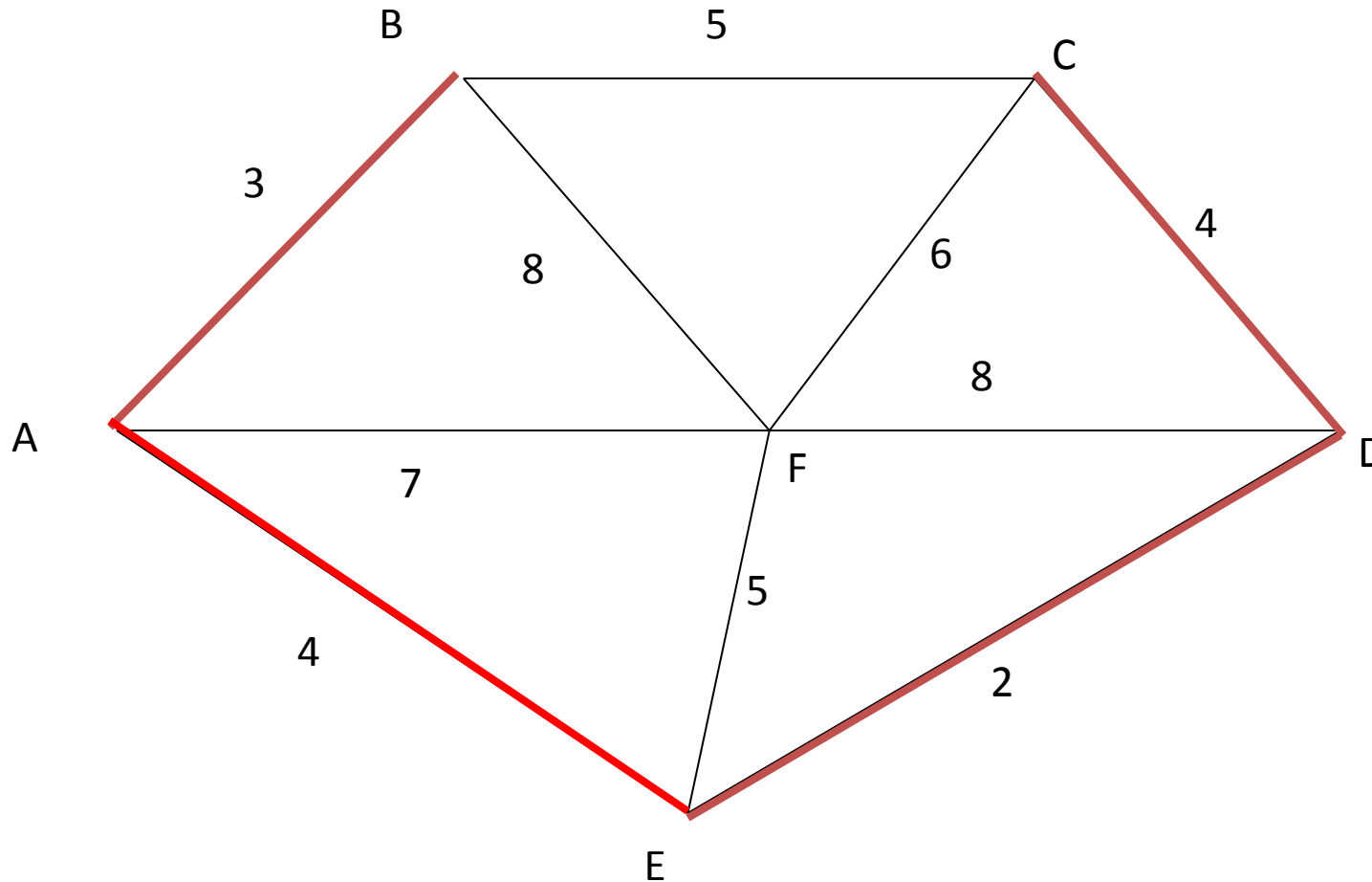
CF 6

AF 7

BF 8

DF 8

Kruskal's Algorithm

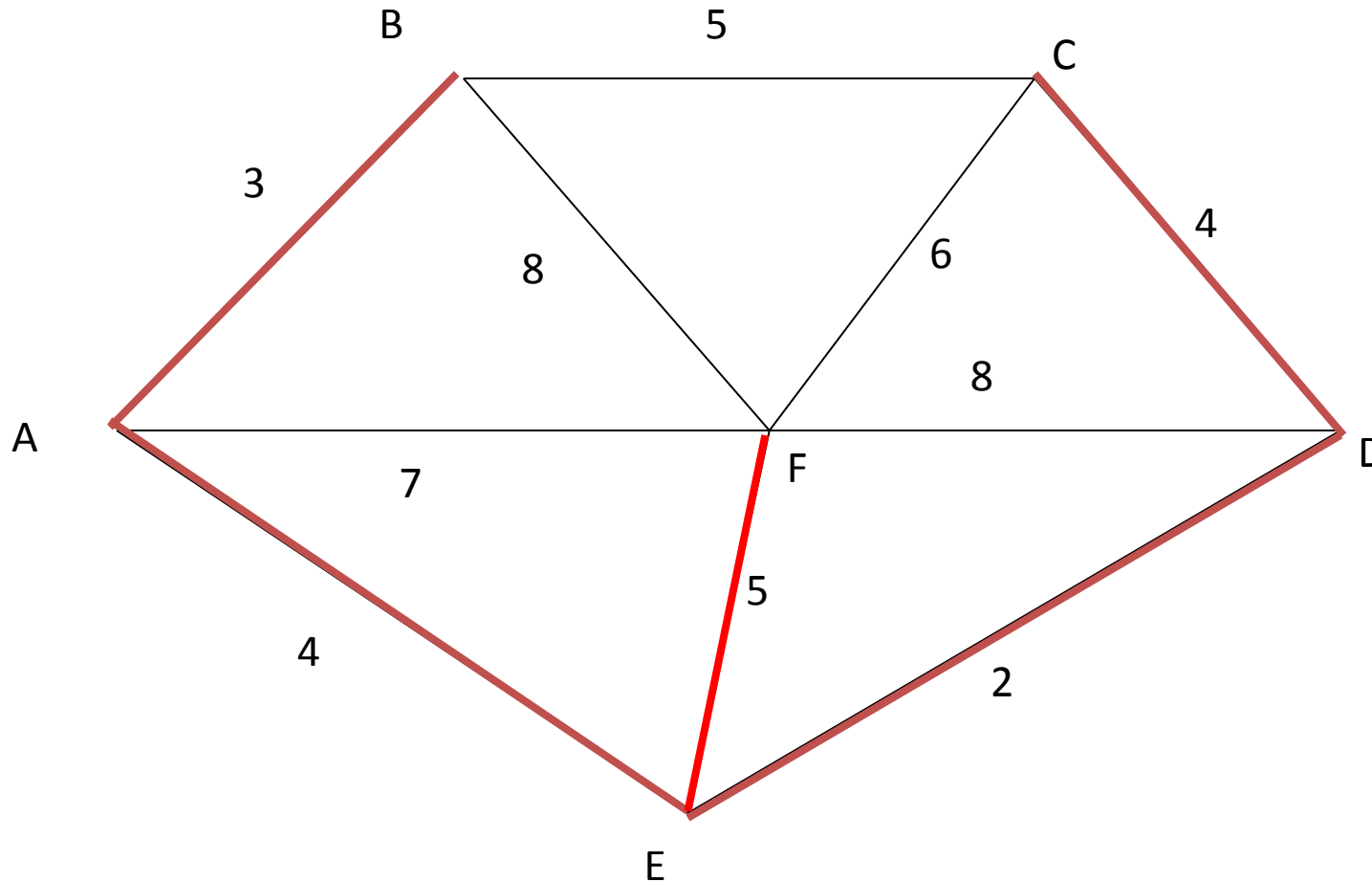


Select the next shortest edge which does not create a cycle

ED 2
AB 3
CD 4
AE 4

ED 2
AB 3
AE 4
CD 4
BC 5
EF 5
CF 6
AF 7
BF 8
DF 8

Kruskal's Algorithm



Select the next shortest edge which does not create a cycle

ED 2

AB 3

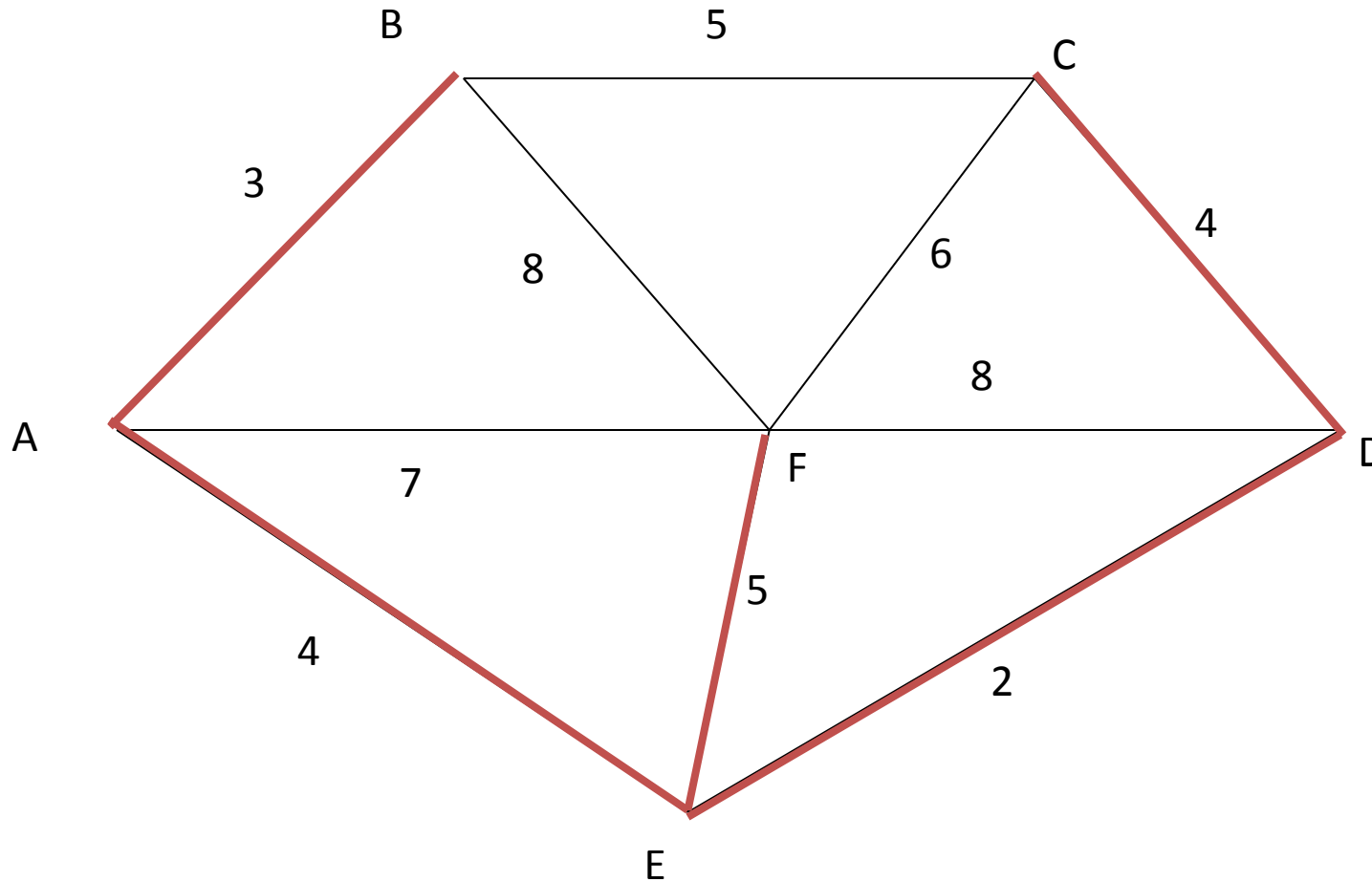
CD 4

AE 4

BC 5 – forms a cycle

EF 5

Kruskal's Algorithm



All vertices have been connected.

The solution is

ED 2

AB 3

CD 4

AE 4

EF 5

Total weight of tree: 18

PRIM'S ALGORITHM

- ✓ Select any vertex
- ✓ Select the shortest edge connected to the vertex.
- ✓ Select the shortest edge connected to any vertex already connected.
- ✓ Repeat step 3 until all vertices have been connected

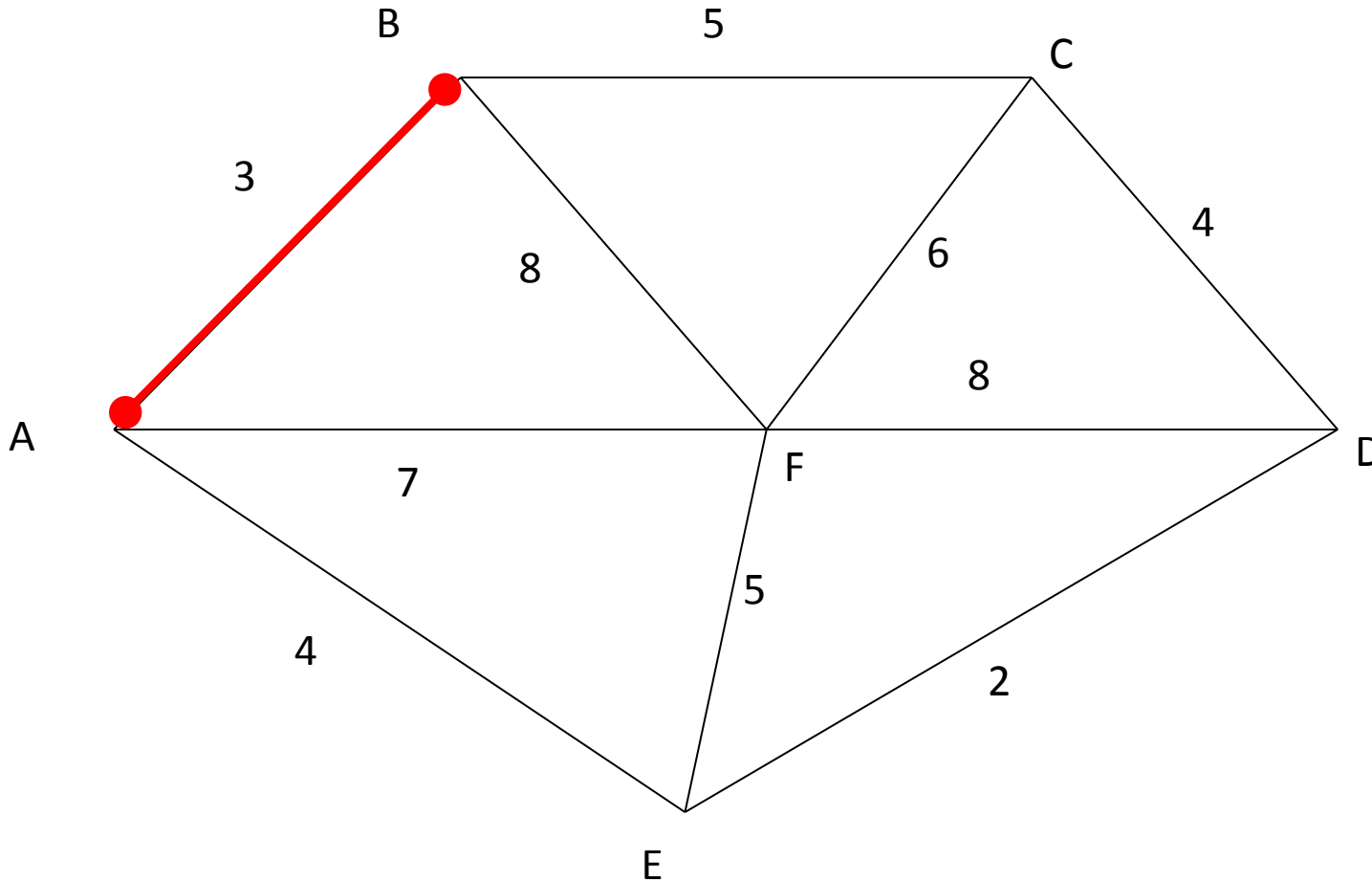
Prim's Algorithm

Select any vertex

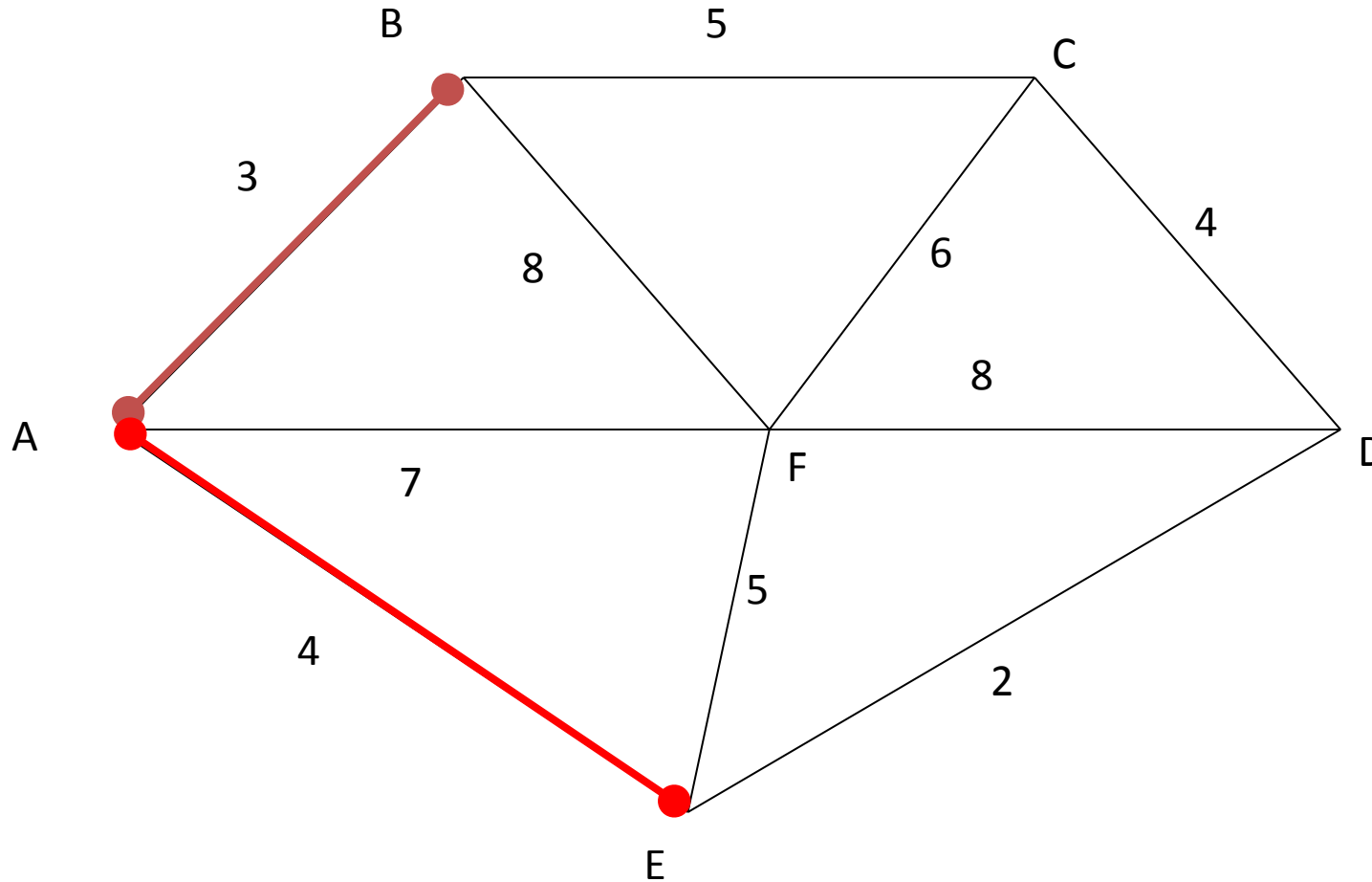
A

Select the shortest edge
connected to that vertex

AB 3



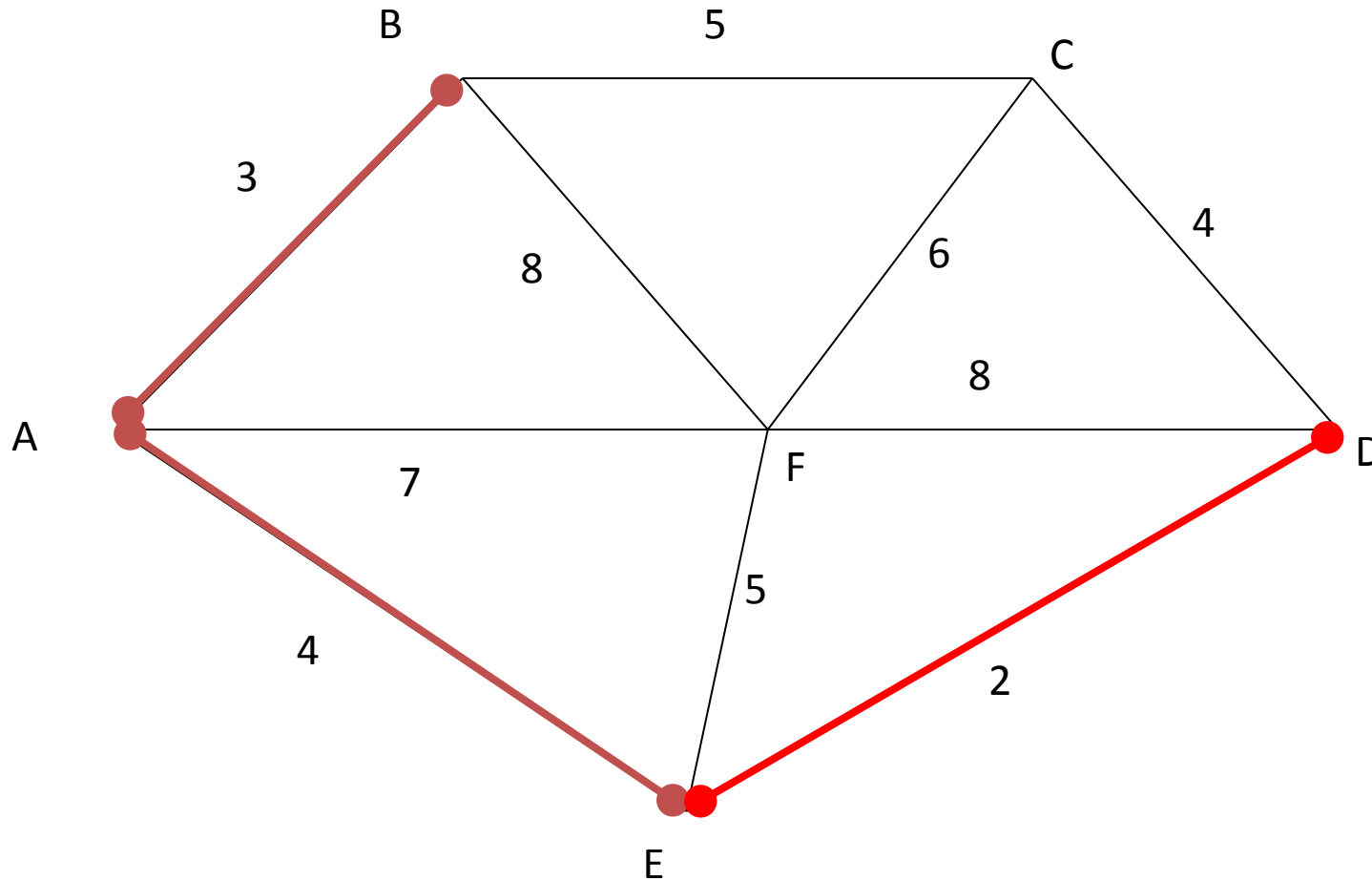
Prim's Algorithm



Select the shortest edge connected to any vertex already connected.

AE 4

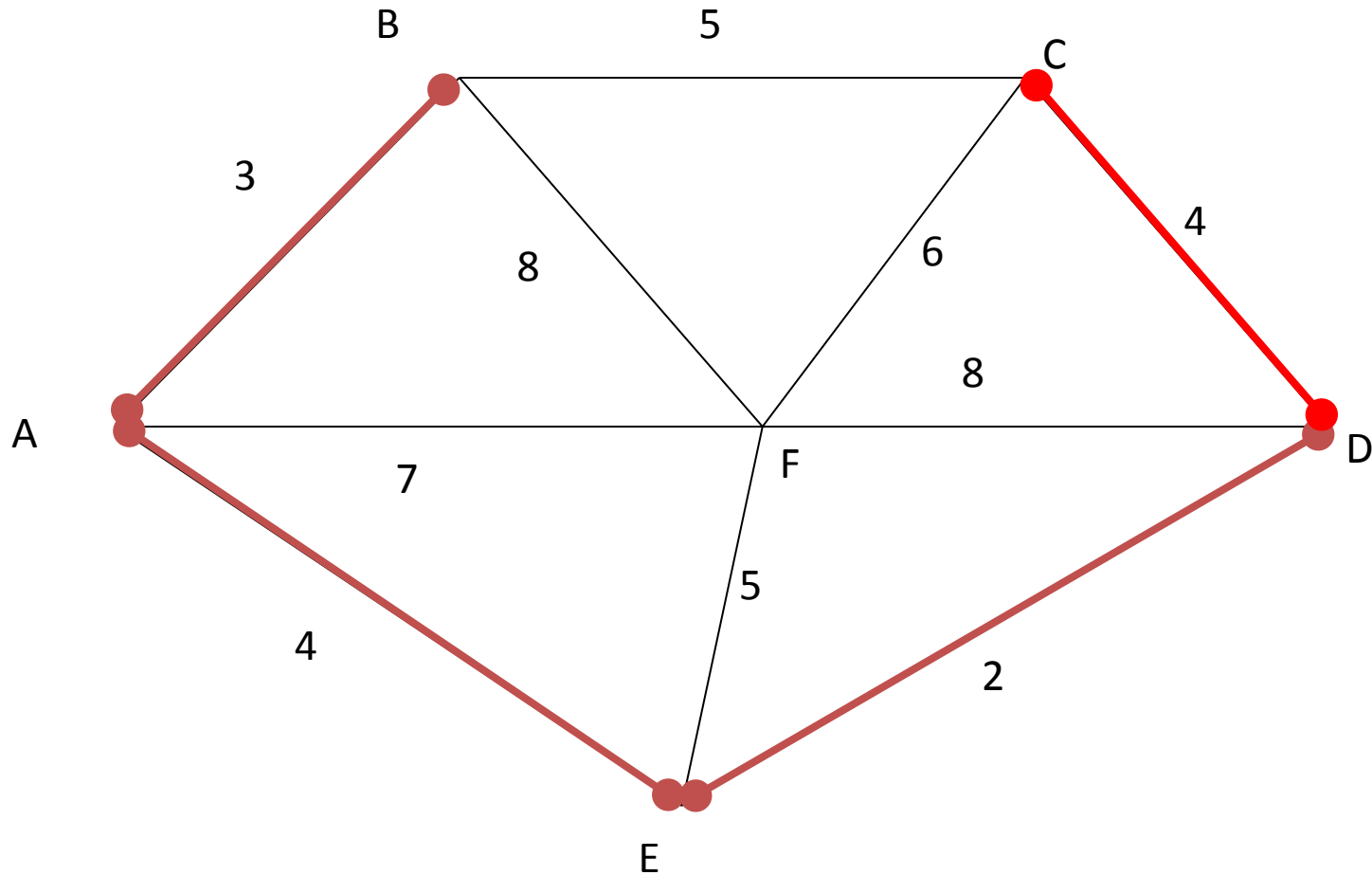
Prim's Algorithm



Select the shortest edge connected to any vertex already connected.

ED 2

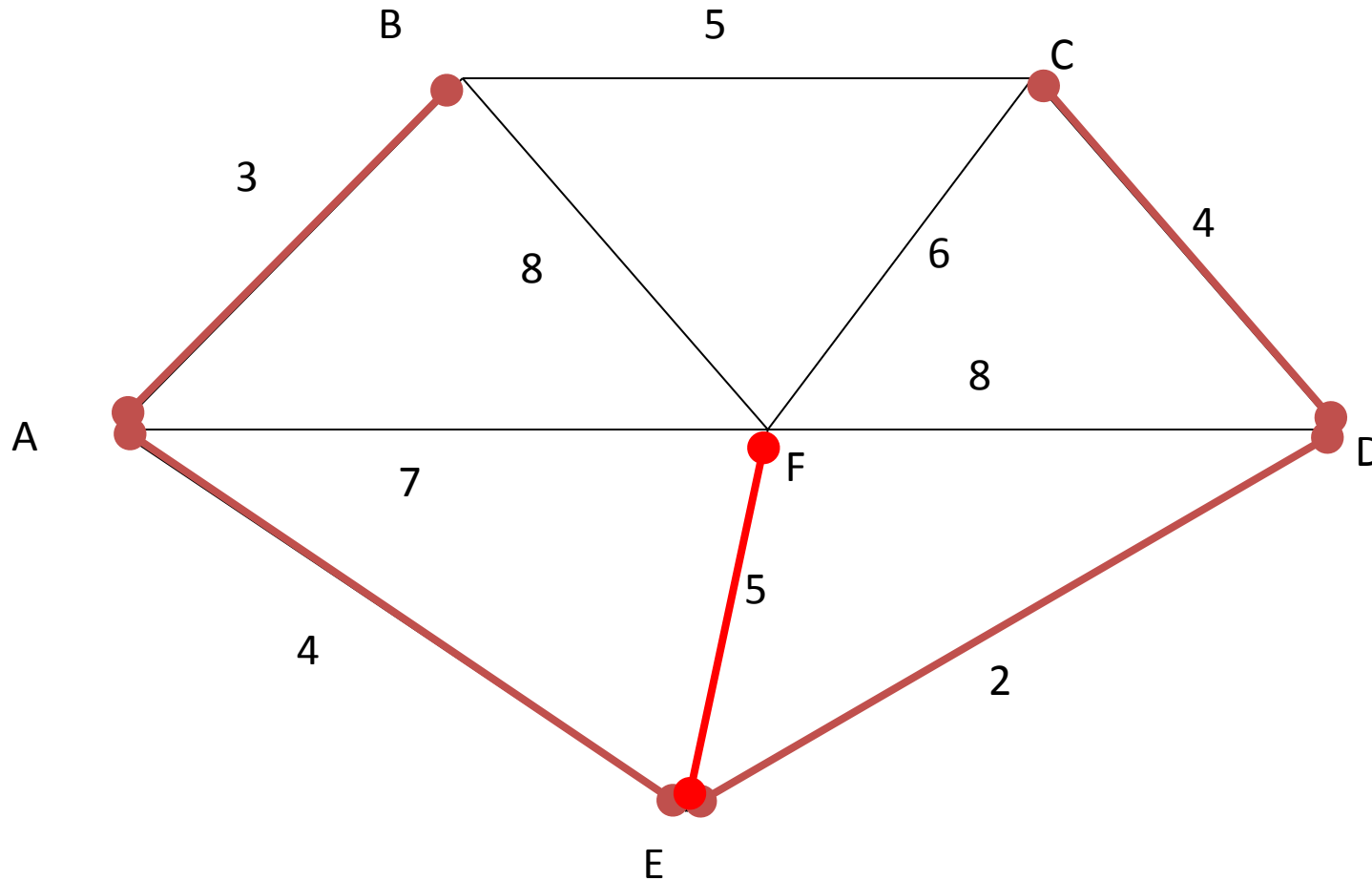
Prim's Algorithm



Select the shortest edge connected to any vertex already connected.

DC 4

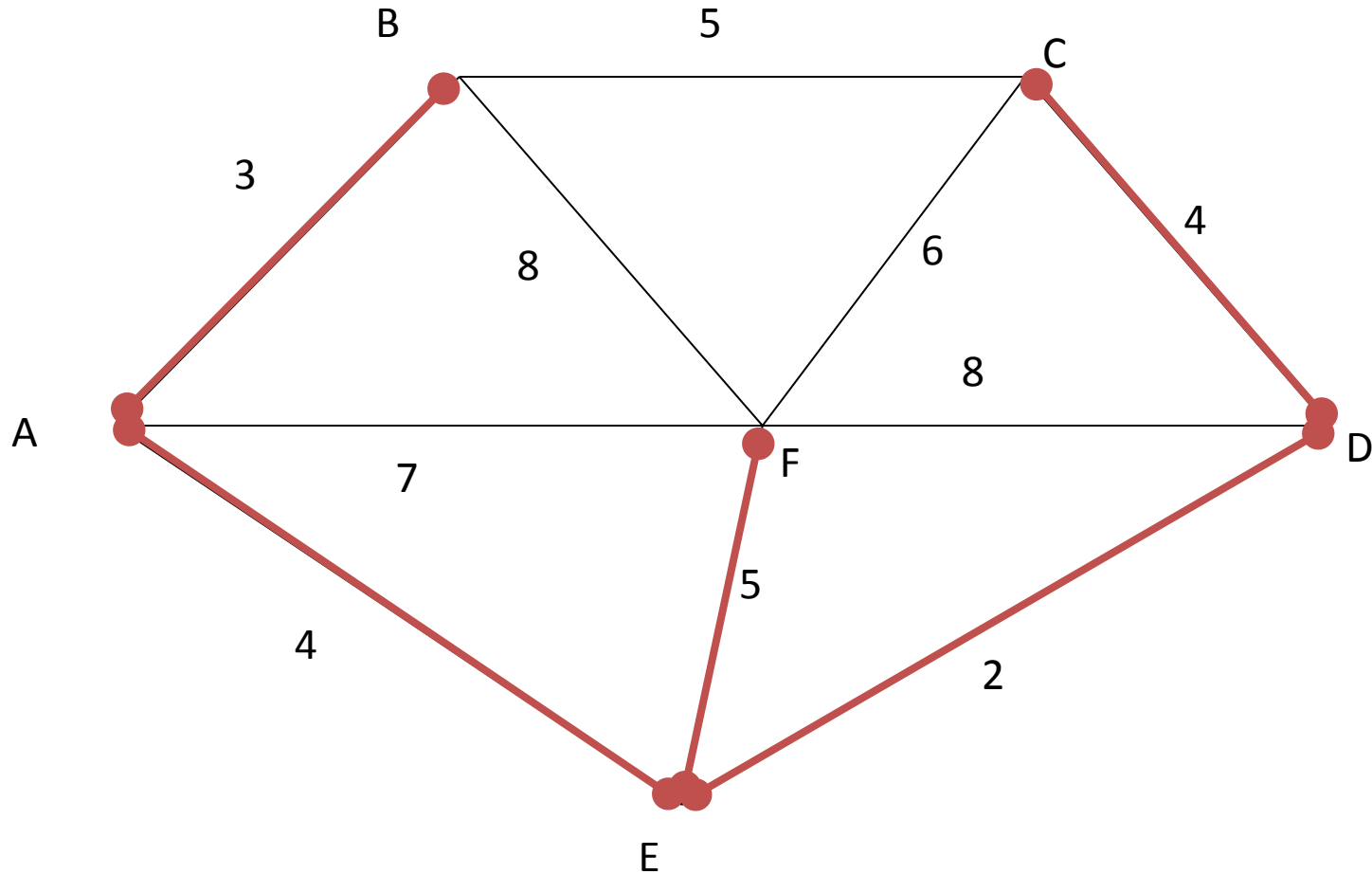
Prim's Algorithm



Select the shortest edge connected to any vertex already connected.

EF 5

Prim's Algorithm



All vertices have been connected.

The solution is

AB 3

AE 4

ED 2

DC 4

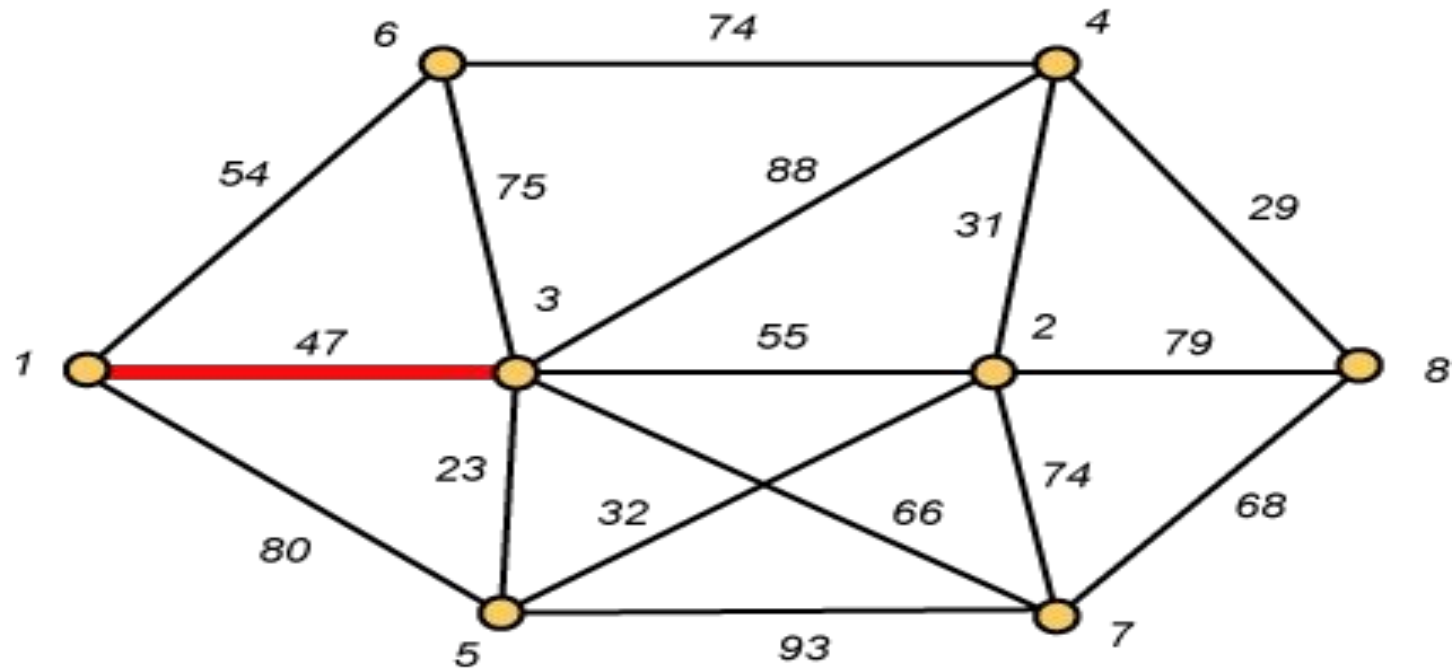
EF 5

Total weight of tree: 18

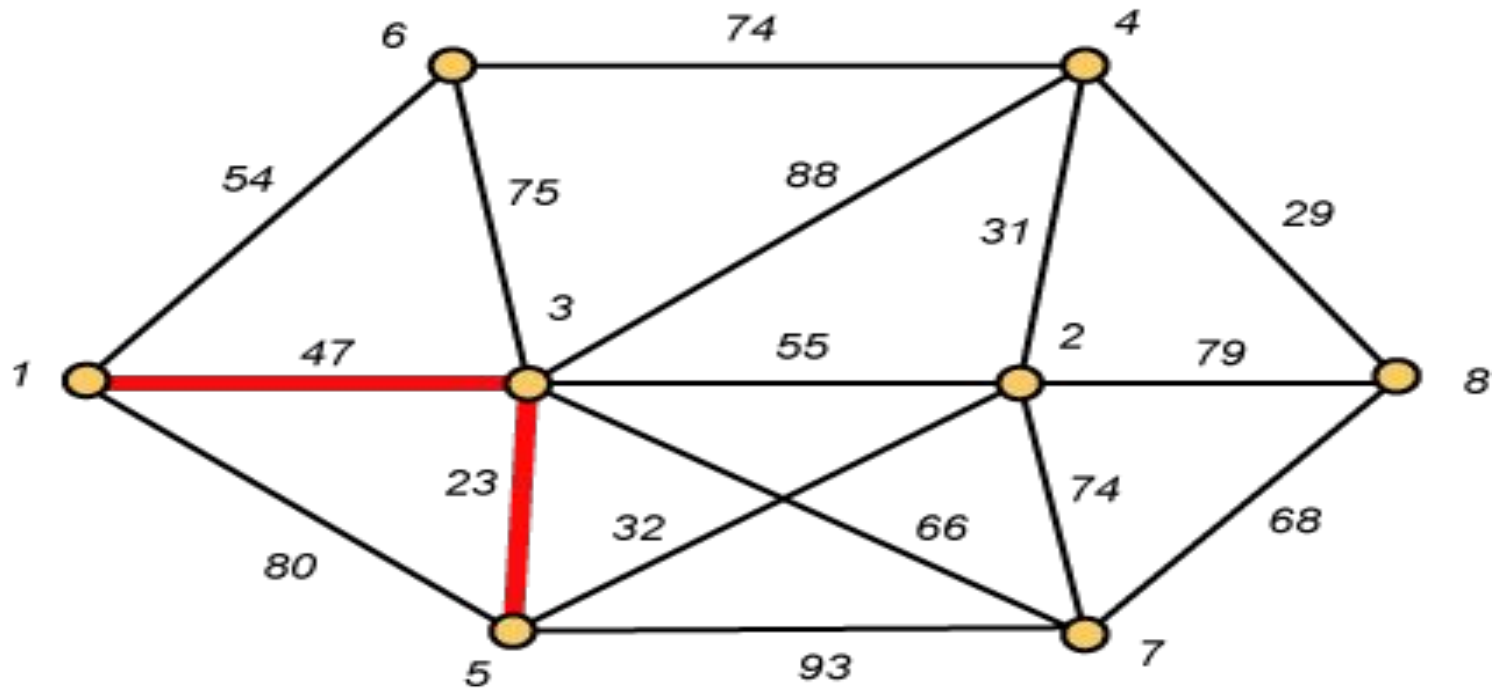
Prim's Algorithm

- Vertex based algorithm
- Grows one tree T , **one vertex at a time**

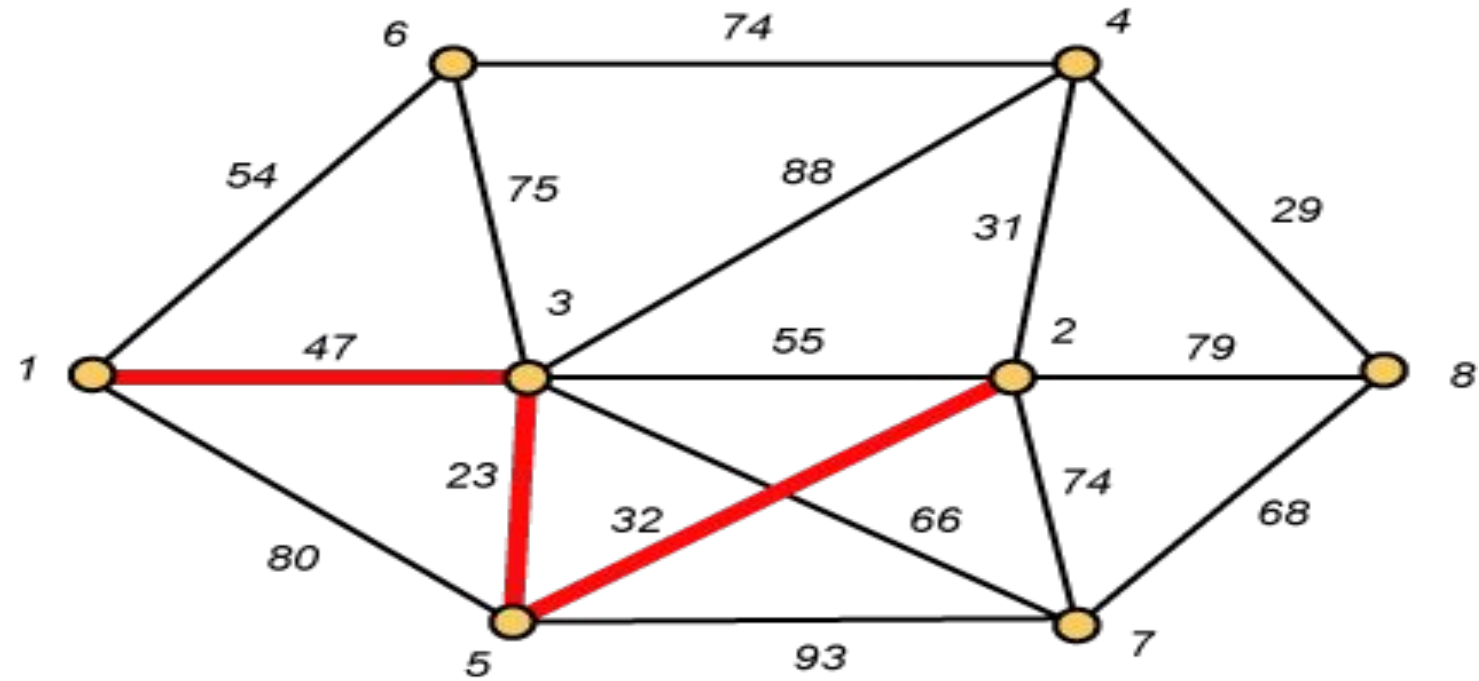
Prim – Step 1



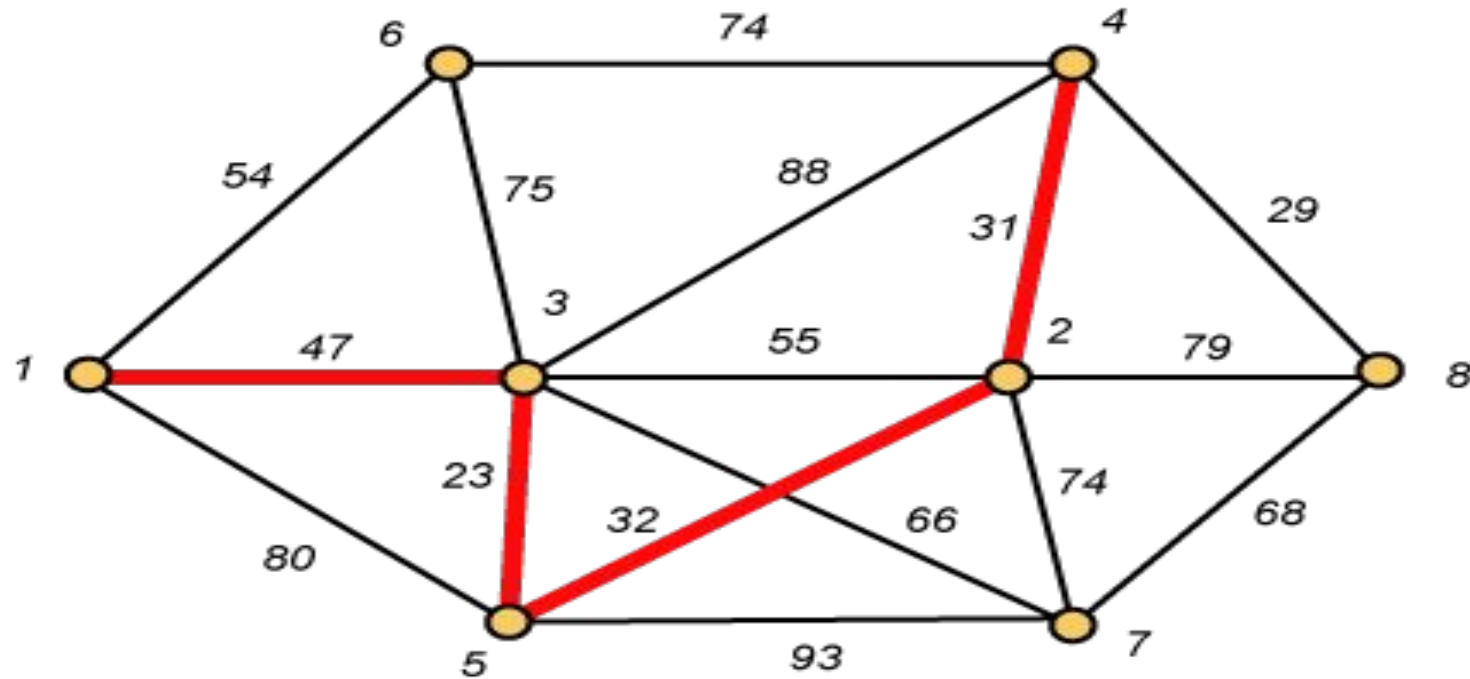
Prim – Step 2



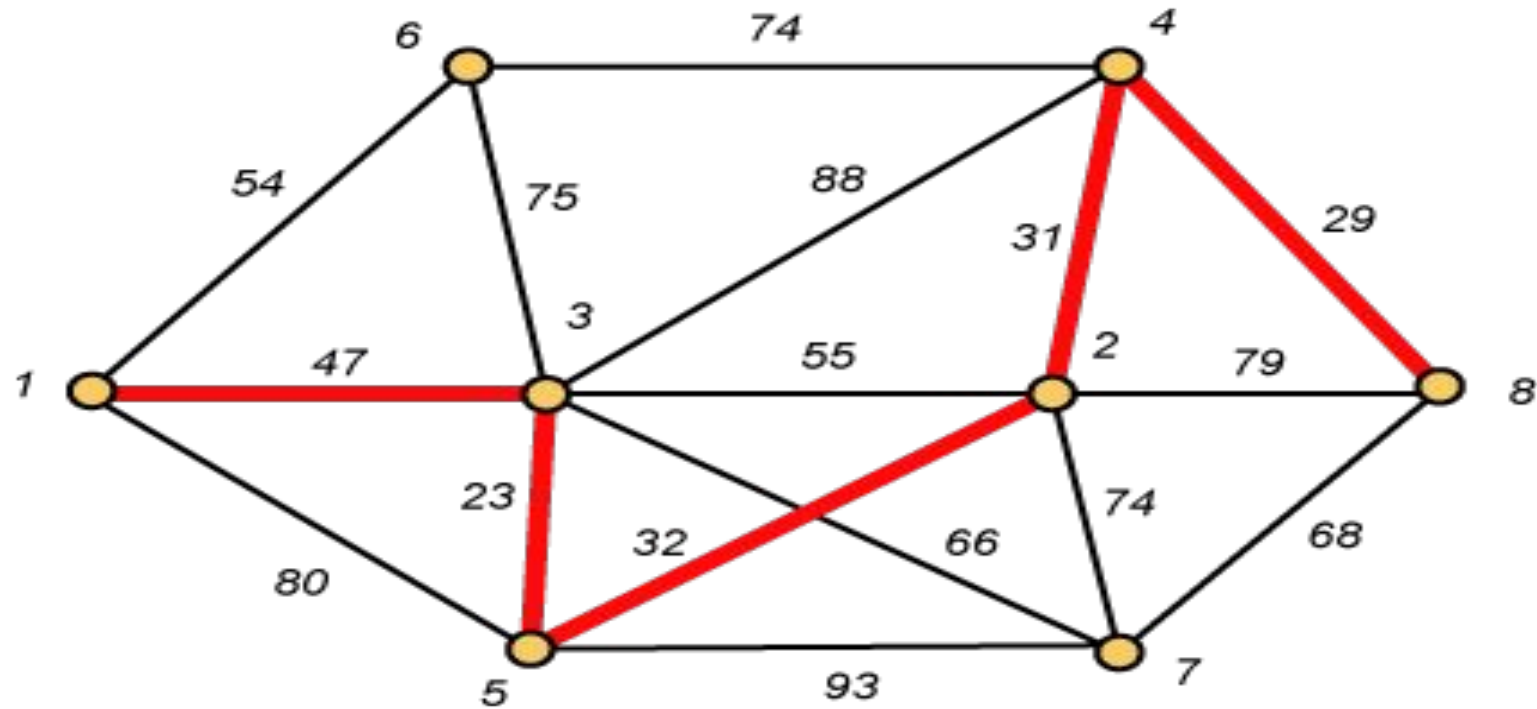
Prim – Step 3



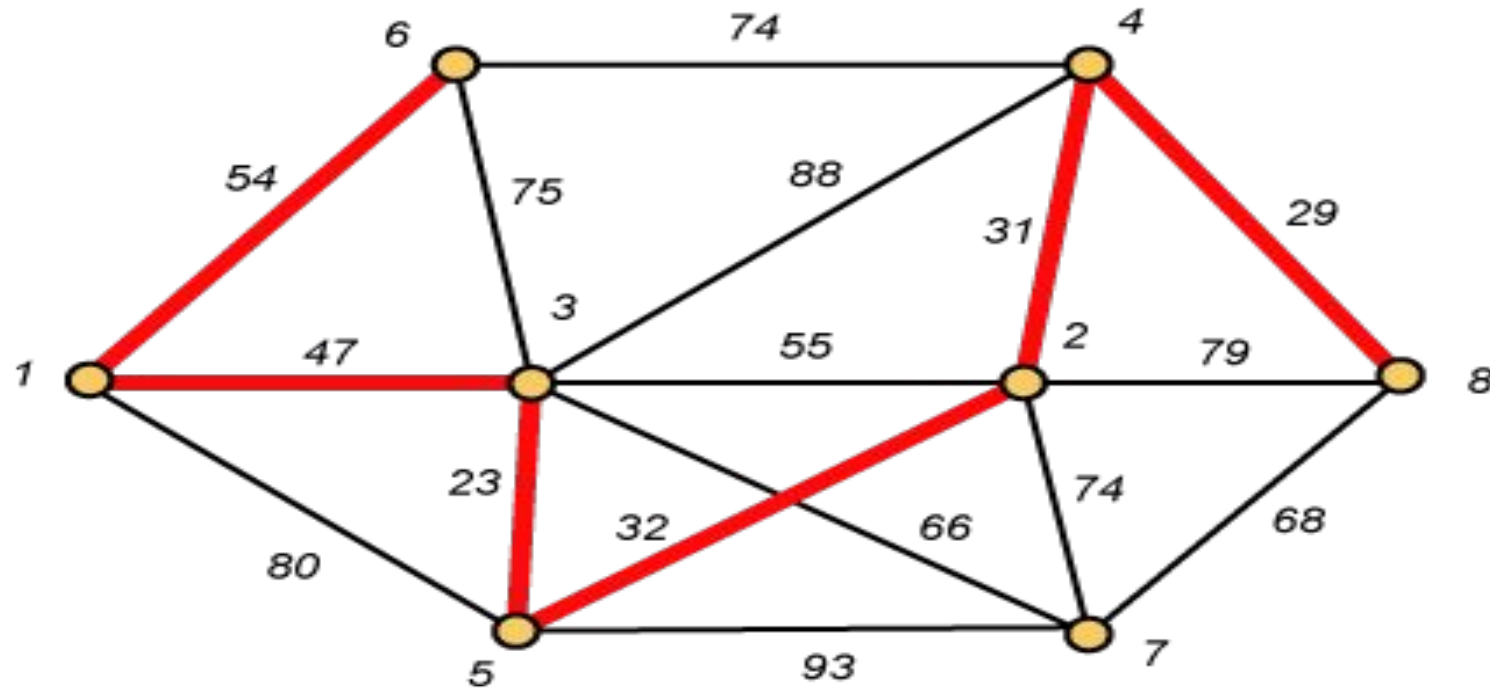
Prim – Step 4



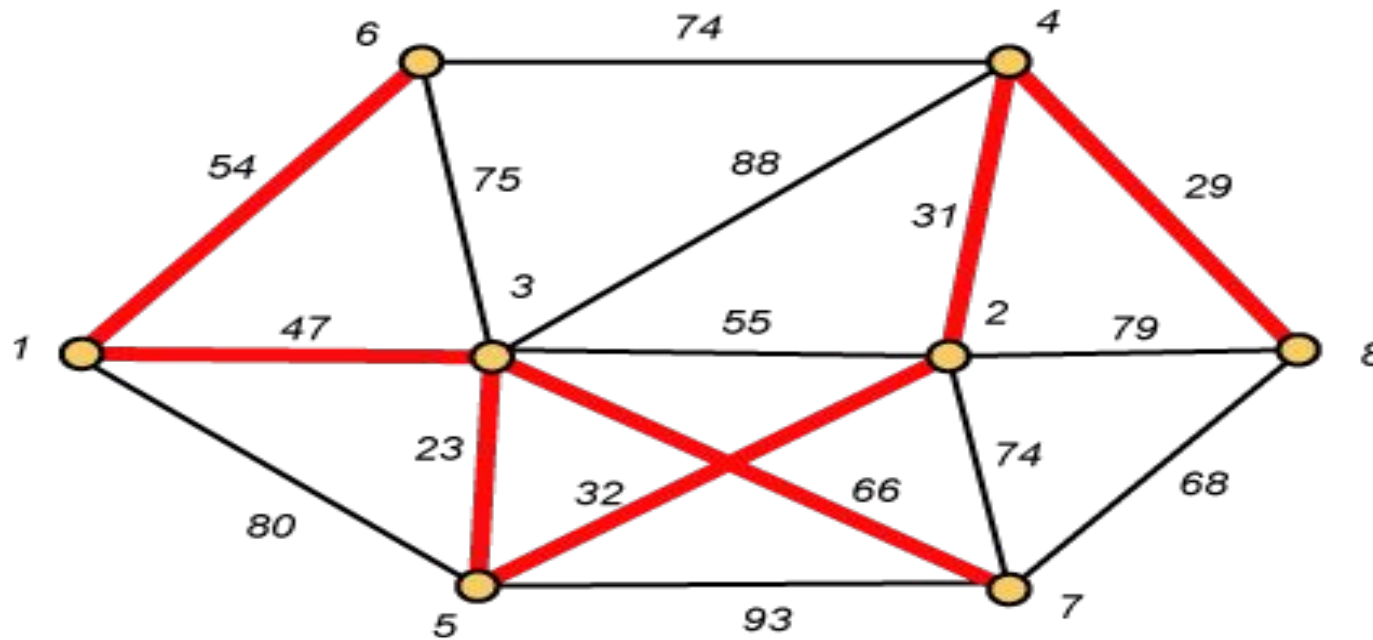
Prim – Step 5



Prim – Step 6



Prim – Step 7 *Done!!*



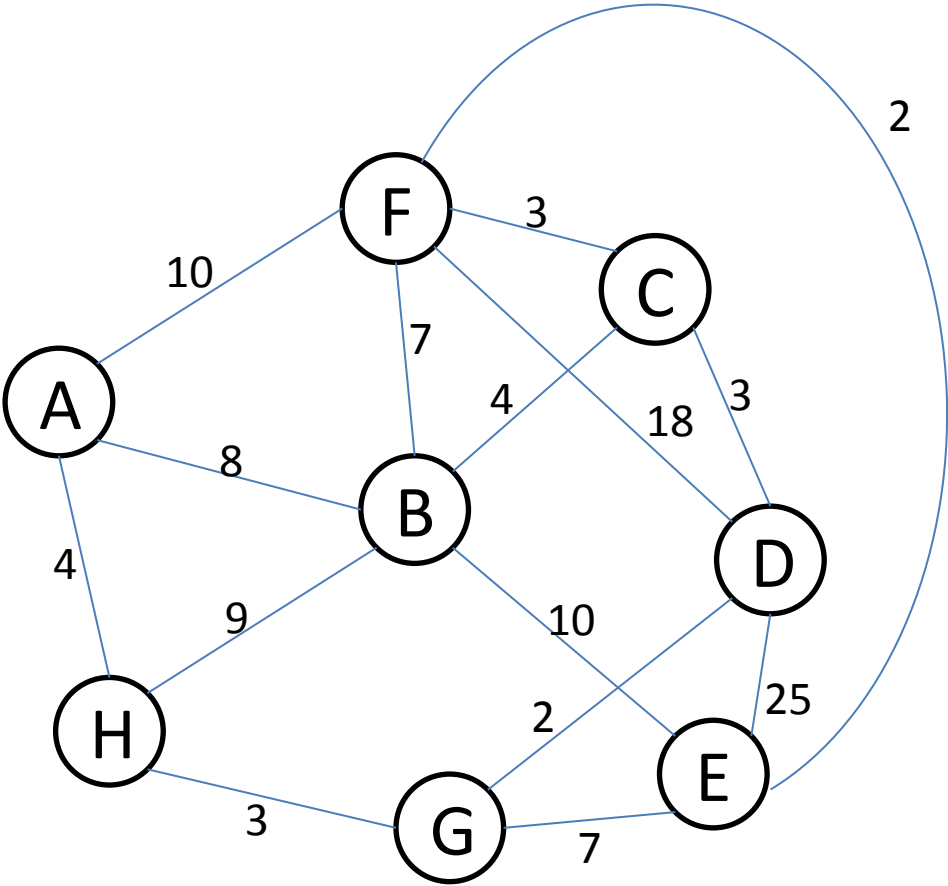
$$\text{Weight (T)} = 23 + 29 + 31 + 32 + 47 + 54 + 66 = \mathbf{282}$$

PRIM'S ALGORITHM

- k_v : has v been visited?
- d_v : What is the minimal edge weight to v ?
- p_v : What vertex precedes (is parent of) v ?

Initialize array

	K	dv	Pv
A	F		-----
B	F		-----
C	F		-----
D	F		-----
E	F		-----
F	F		-----
G	F		-----
H	F		-----



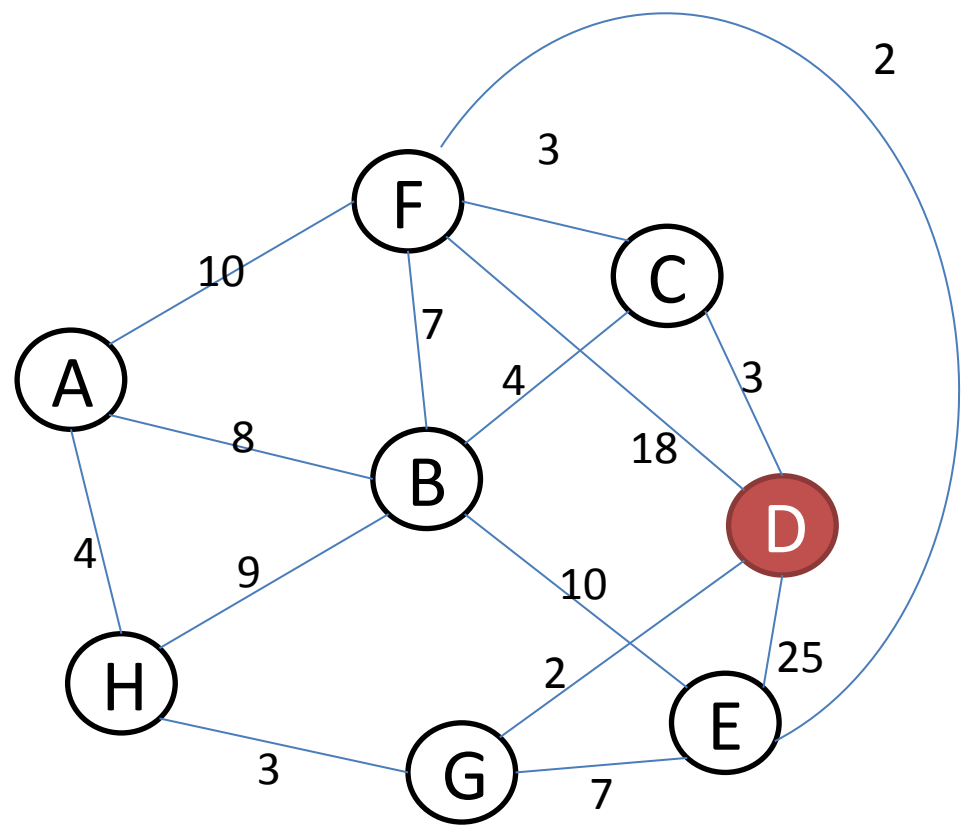
PRIM'S ALGORITHM

– k_v : has v been visited?

– d_v : What is the minimal edge weight to v ?

– p_v : What vertex precedes (is parent of) v ?

Start with any node, say D

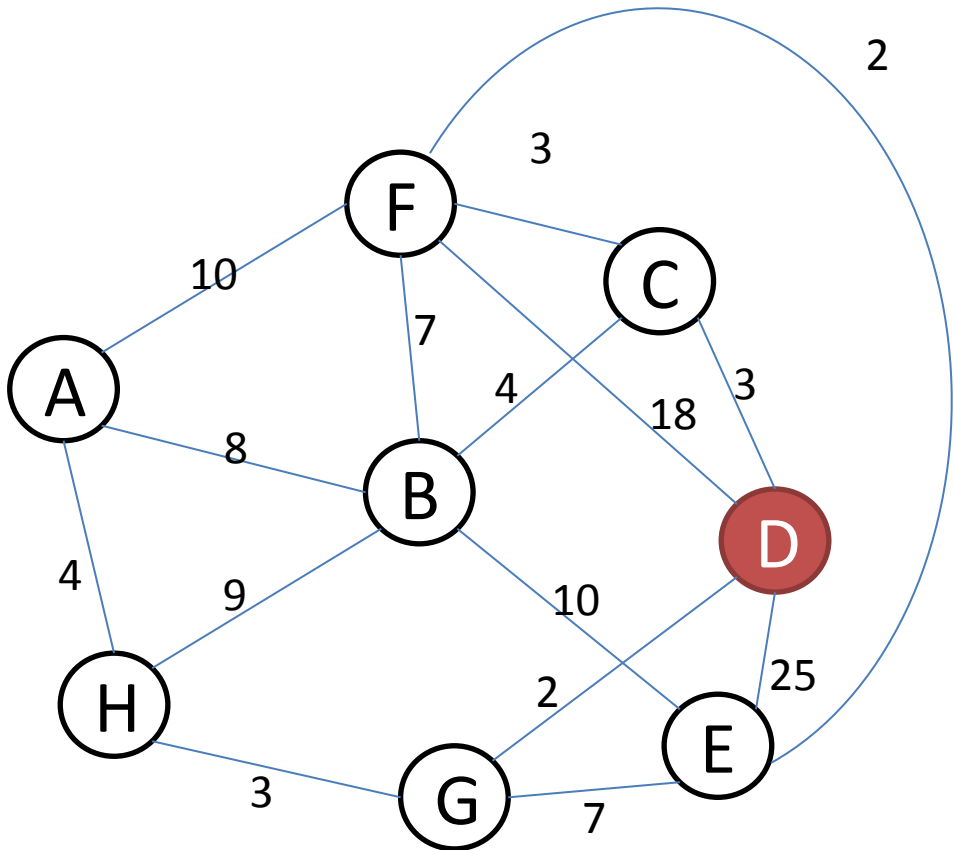


	K	d_v	P_v
A			
B			
C			
D	T	0	--
E			
F			
G			
H			

PRIM'S ALGORITHM

- k_v : has v been visited?
- d_v : What is the minimal edge weight to v ?
- p_v : What vertex precedes (is parent of) v ?

Update distances of adjacent, unselected nodes

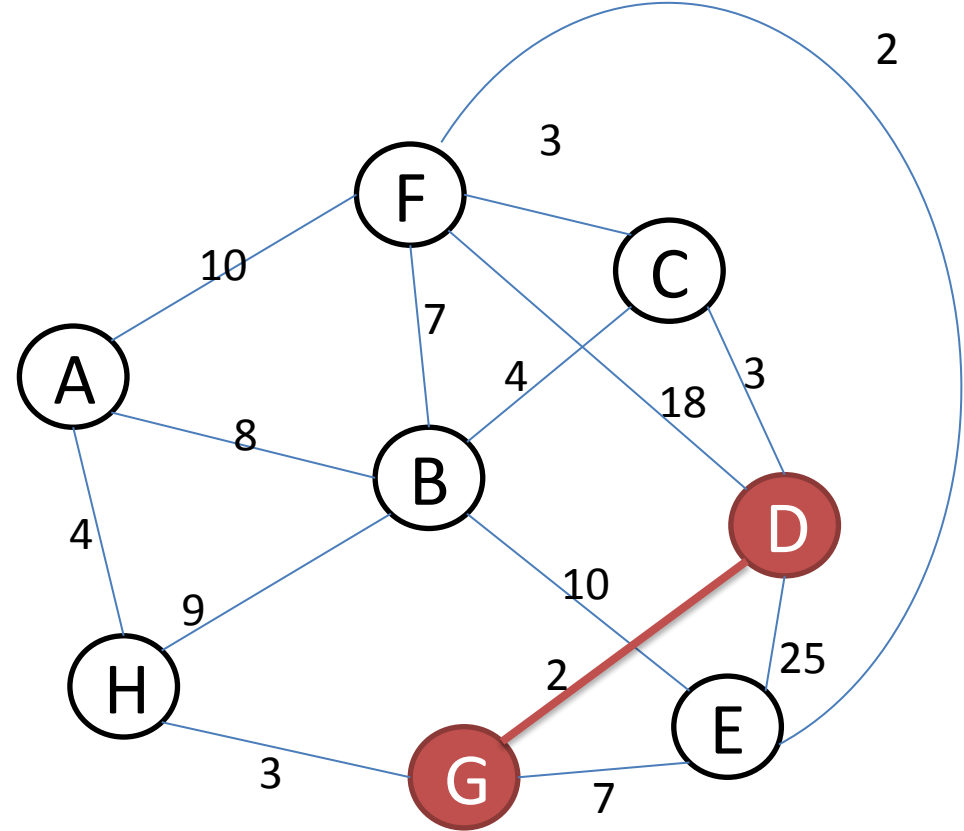


	K	d_v	P_v
A			
B			
C		3	D
D	T	0	--
E		25	D
F		18	D
G		2	D
H			

PRIM'S ALGORITHM

- k_v : has v been visited?
- d_v : What is the minimal edge weight to v ?
- p_v : What vertex precedes (is parent of) v ?

Select node with minimum distance

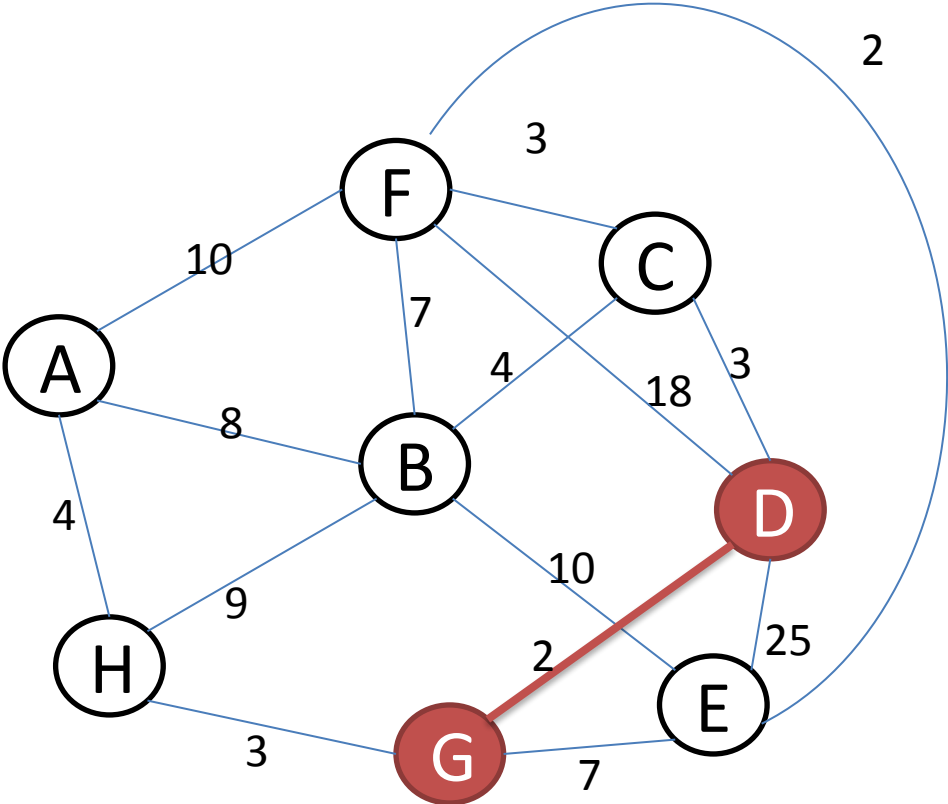


	K	d_v	P_v
A			
B			
C		3	D
D	T	0	--
E		25	D
F		18	D
G	T	2	D
H			

PRIM'S ALGORITHM

- k_v : has v been visited?
- d_v : What is the minimal edge weight to v ?
- p_v : What vertex precedes (is parent of) v ?

Update distances of adjacent, unselected nodes



	K	d_v	P_v
A			
B			
C		3	D
D	T	0	--
E		7	G
F		18	D
G	T	2	D
H		3	G

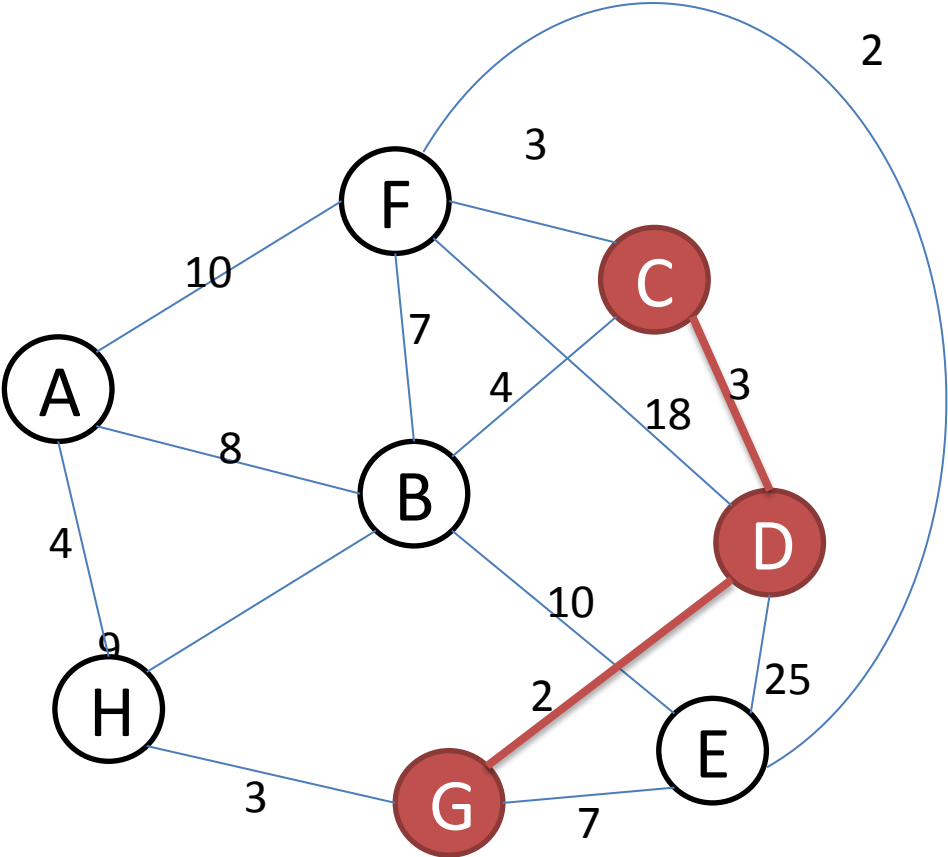
PRIM'S ALGORITHM

– k_v : has v been visited?

– d_v : What is the minimal edge weight to v ?

– p_v : What vertex precedes (is parent of) v ?

Select node with minimum distance



	K	d_v	P_v
A			
B			
C	T	3	D
D	T	0	--
E		7	G
F		18	D
G	T	2	D
H		3	G

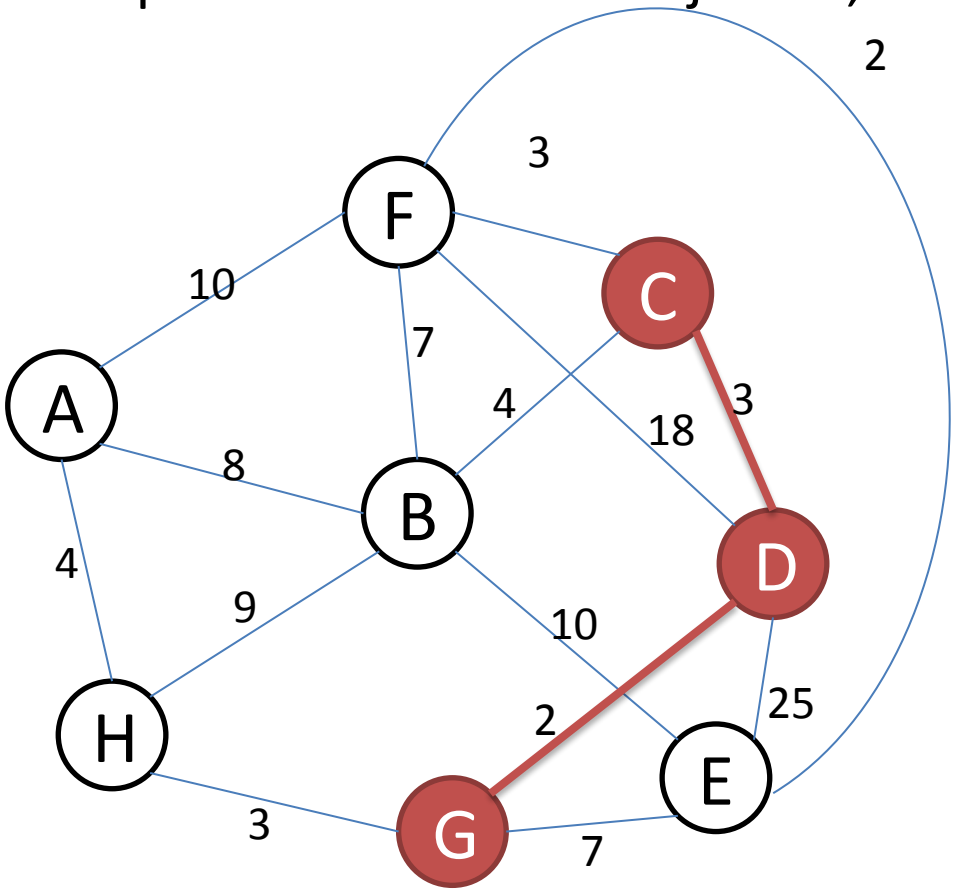
PRIM'S ALGORITHM

– k_v : has v been visited?

– d_v : What is the minimal edge weight to v ?

– p_v : What vertex precedes (is parent of) v ?

Update distances of adjacent, unselected nodes



	K	d_v	P_v
A			
B		4	C
C	T	3	D
D	T	0	--
E		7	G
F		3	C
G	T	2	D
H		3	G

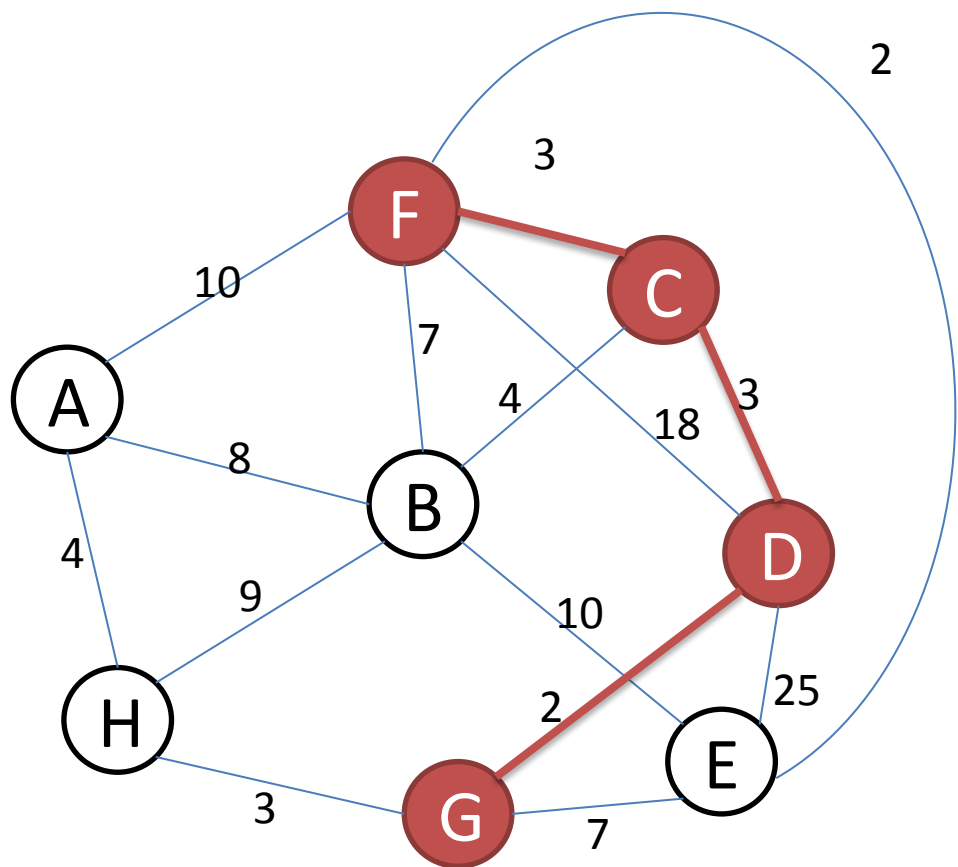
PRIM'S ALGORITHM

– k_v : has v been visited?

– d_v : What is the minimal edge weight to v ?

– p_v : What vertex precedes (is parent of) v ?

Select node with minimum distance



	K	d_v	P_v
A			
B		4	C
C	T	3	D
D	T	0	--
E		7	G
F	T	3	C
G	T	2	D
H		3	G

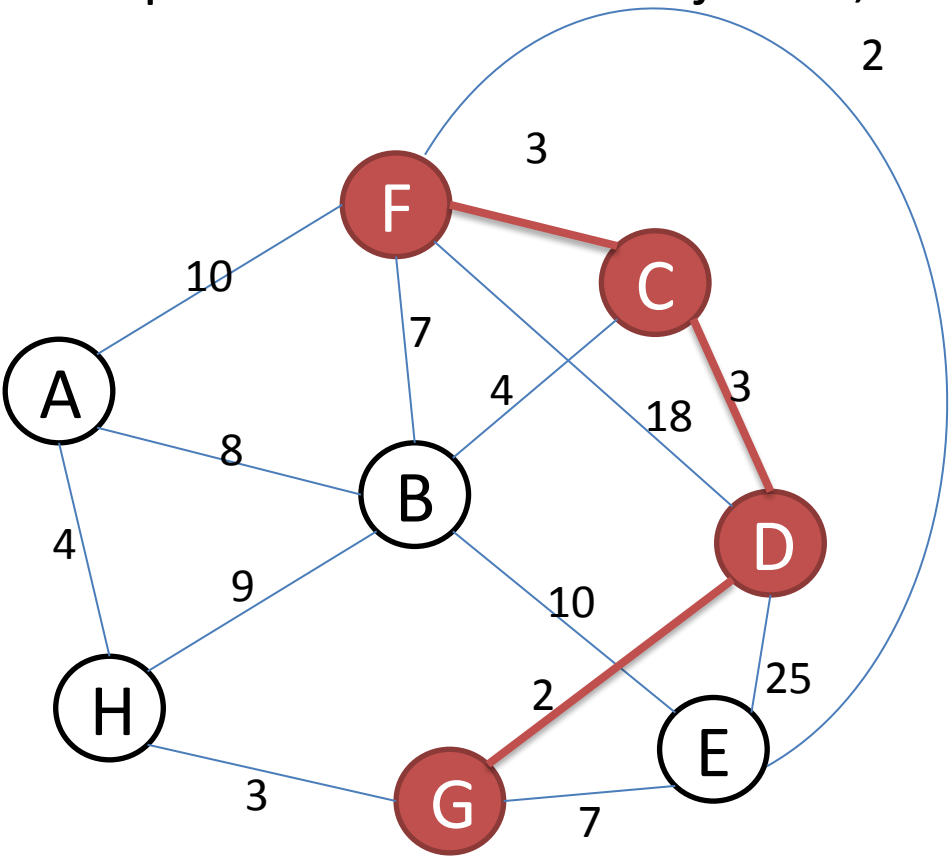
PRIM'S ALGORITHM

– k_v : has v been visited?

– d_v : What is the minimal edge weight to v ?

– p_v : What vertex precedes (is parent of) v ?

Update distances of adjacent, unselected nodes



	K	d_v	P_v
A		10	F
B		4	C
C	T	3	D
D	T	0	--
E		2	F
F	T	3	C
G	T	2	D
H		3	G

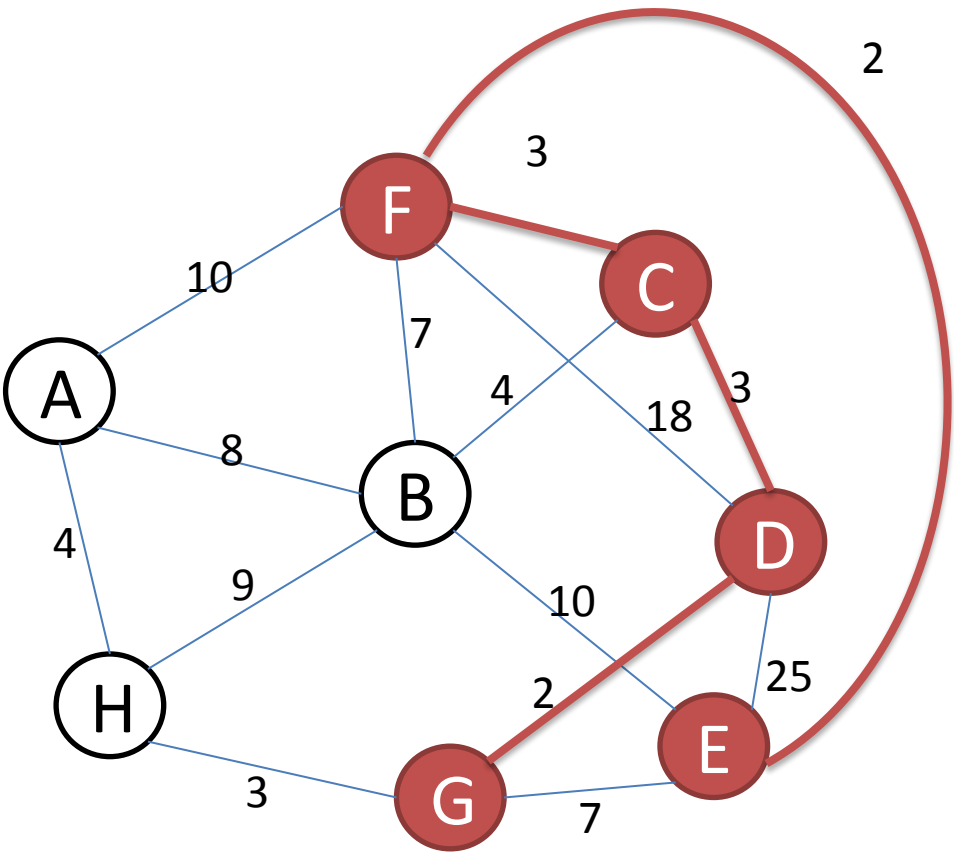
PRIM'S ALGORITHM

– k_v : has v been visited?

– d_v : What is the minimal edge weight to v ?

– p_v : What vertex precedes (is parent of) v ?

Select node with minimum distance



	K	d_v	P_v
A		10	F
B		4	C
C	T	3	D
D	T	0	--
E	T	2	F
F	T	3	C
G	T	2	D
H		3	G

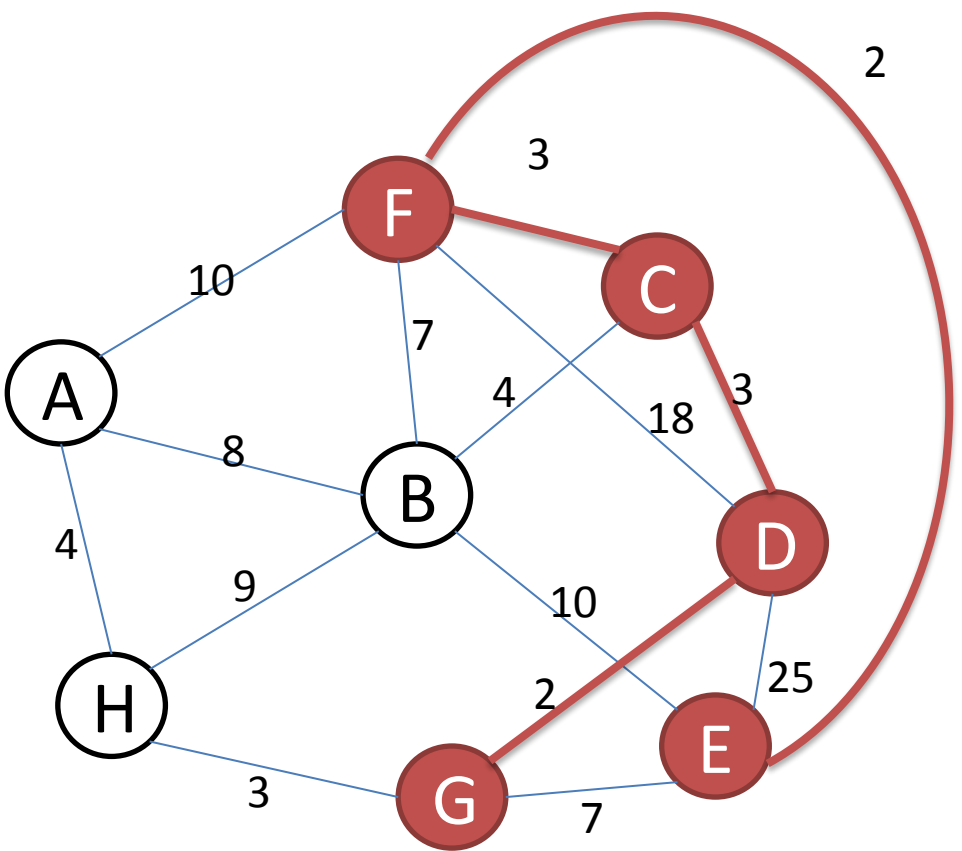
PRIM'S ALGORITHM

– k_v : has v been visited?

– d_v : What is the minimal edge weight to v ?

– p_v : What vertex precedes (is parent of) v ?

Update distances of adjacent, unselected nodes



	K	d_v	P_v
A		10	F
B		4	C
C	T	3	D
D	T	0	--
E	T	2	F
F	T	3	C
G	T	2	D
H		3	G

Table entries unchanged

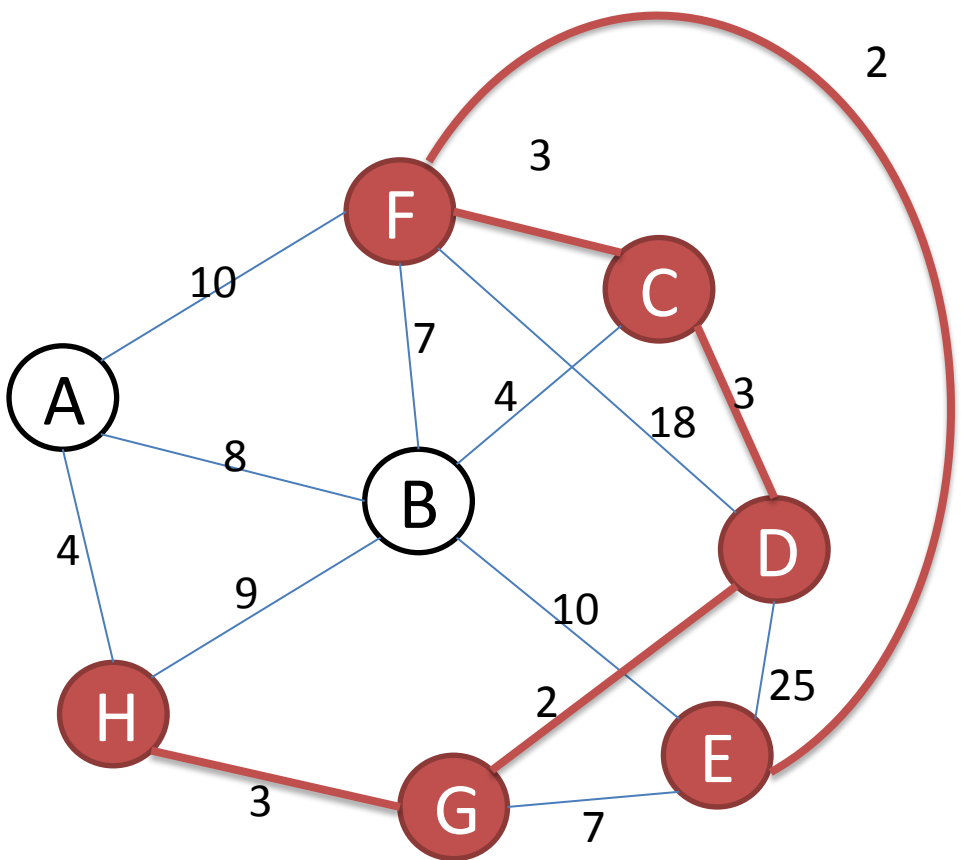
PRIM'S ALGORITHM

– k_v : has v been visited?

– d_v : What is the minimal edge weight to v ?

– p_v : What vertex precedes (is parent of) v ?

Select node with minimum distance



	K	d_v	P_v
A		10	F
B		4	C
C	T	3	D
D	T	0	--
E	T	2	F
F	T	3	C
G	T	2	D
H	T	3	G

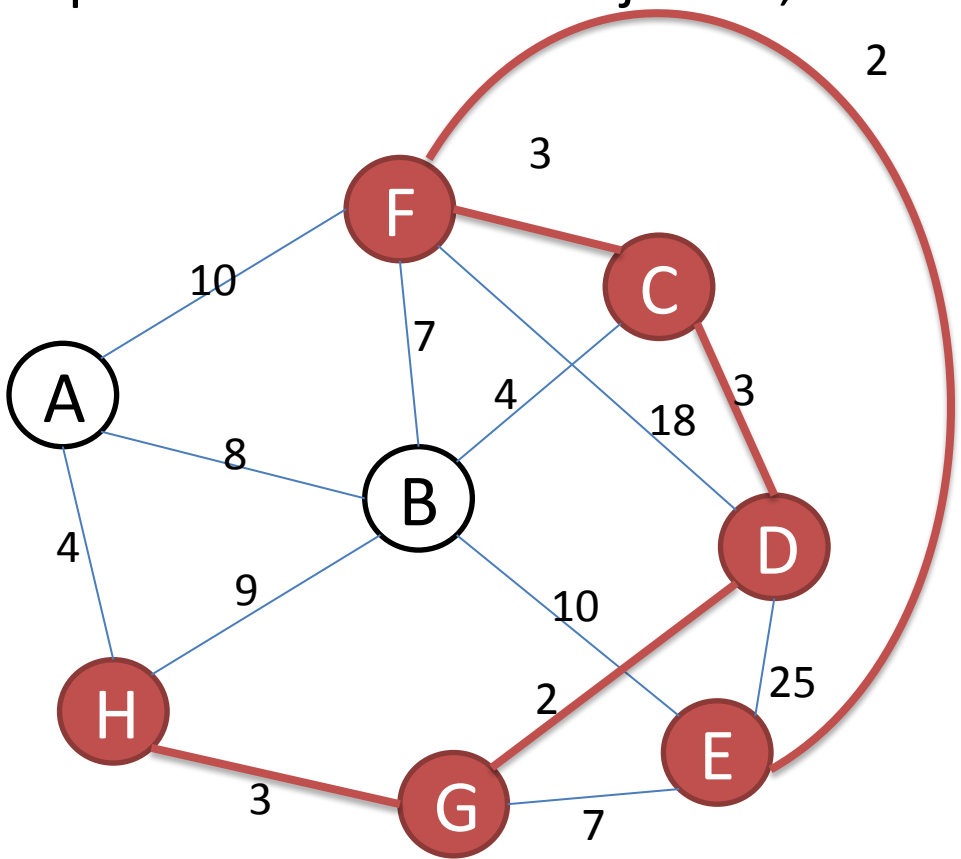
PRIM'S ALGORITHM

– k_v : has v been visited?

– d_v : What is the minimal edge weight to v ?

– p_v : What vertex precedes (is parent of) v ?

Update distances of adjacent, unselected nodes



	K	d_v	P_v
A		4	H
B		4	C
C	T	3	D
D	T	0	--
E	T	2	F
F	T	3	C
G	T	2	D
H	T	3	G

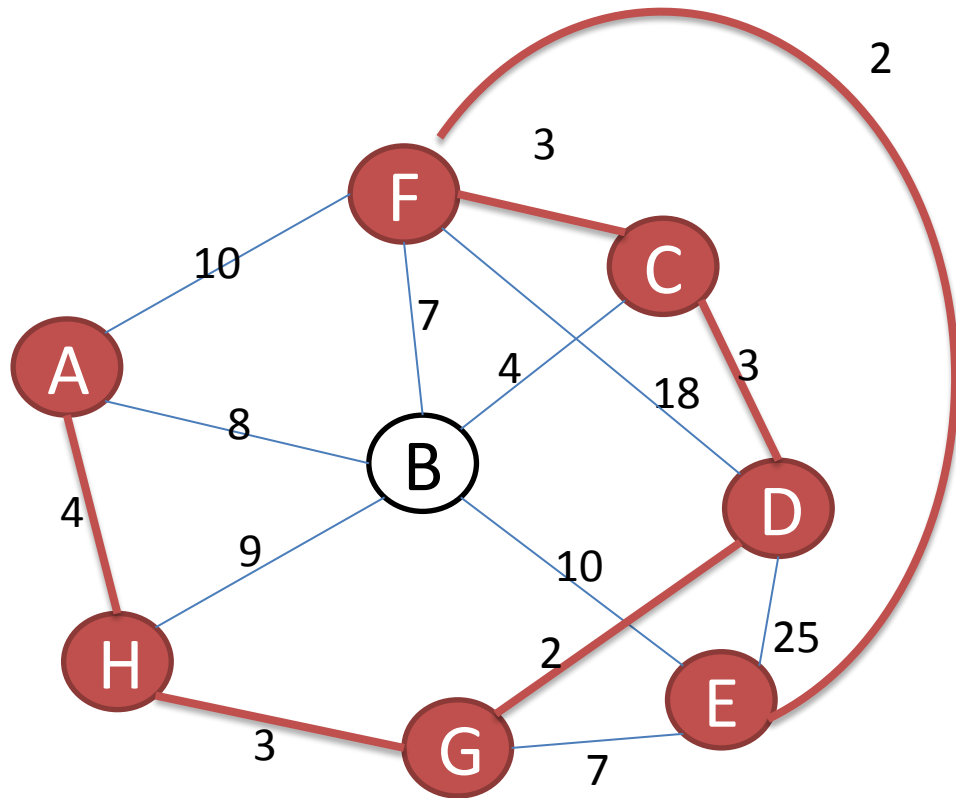
PRIM'S ALGORITHM

– k_v : has v been visited?

– d_v : What is the minimal edge weight to v ?

– p_v : What vertex precedes (is parent of) v ?

Select node with minimum distance



	K	d_v	P_v
A	T	4	H
B		4	C
C	T	3	D
D	T	0	--
E	T	2	F
F	T	3	C
G	T	2	D
H	T	3	G

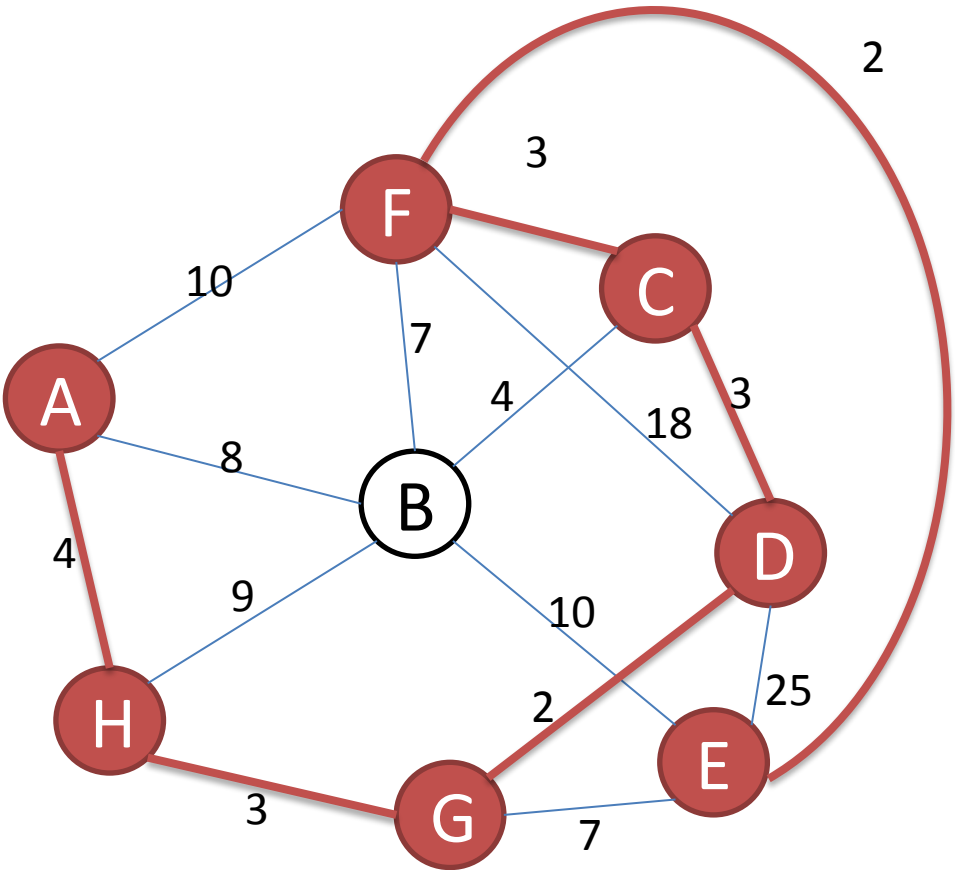
PRIM'S ALGORITHM

– k_v : has v been visited?

– d_v : What is the minimal edge weight to v ?

– p_v : What vertex precedes (is parent of) v ?

Update distances of adjacent, unselected nodes



	K	d_v	P_v
A	T	4	H
B		4	C
C	T	3	D
D	T	0	--
E	T	2	F
F	T	3	C
G	T	2	D
H	T	3	G

Table entries unchanged

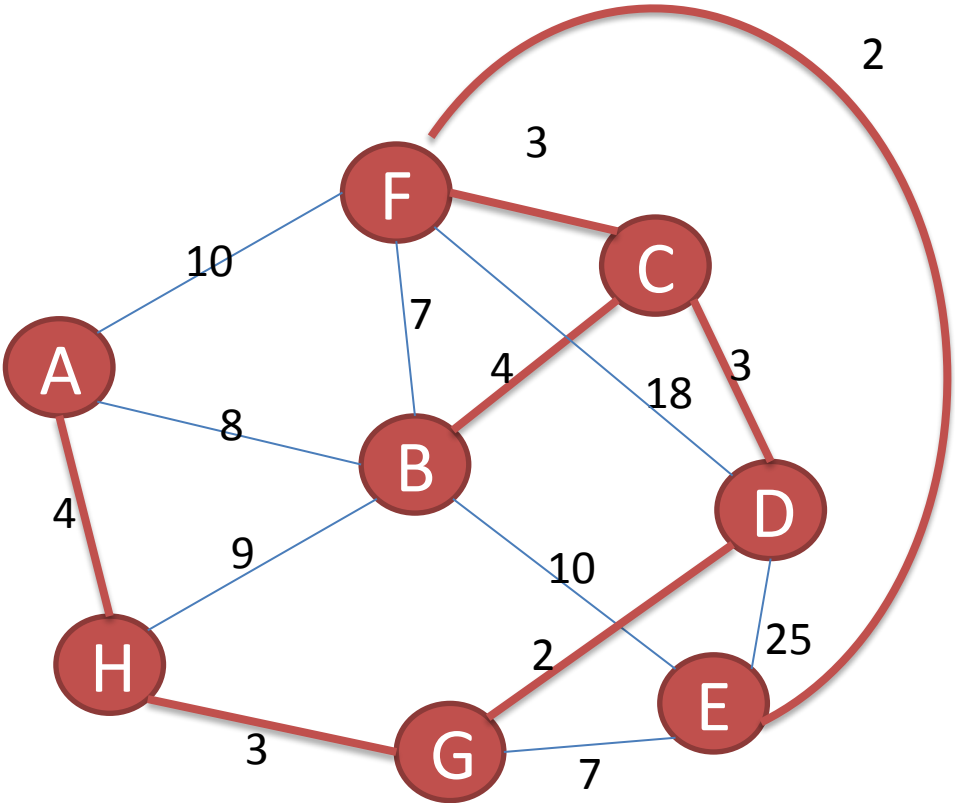
PRIM'S ALGORITHM

– k_v : has v been visited?

– d_v : What is the minimal edge weight to v ?

– p_v : What vertex precedes (is parent of) v ?

Select node with minimum distance



	K	d_v	P_v
A	T	4	H
B	T	4	C
C	T	3	D
D	T	0	--
E	T	2	F
F	T	3	C
G	T	2	D
H	T	3	G

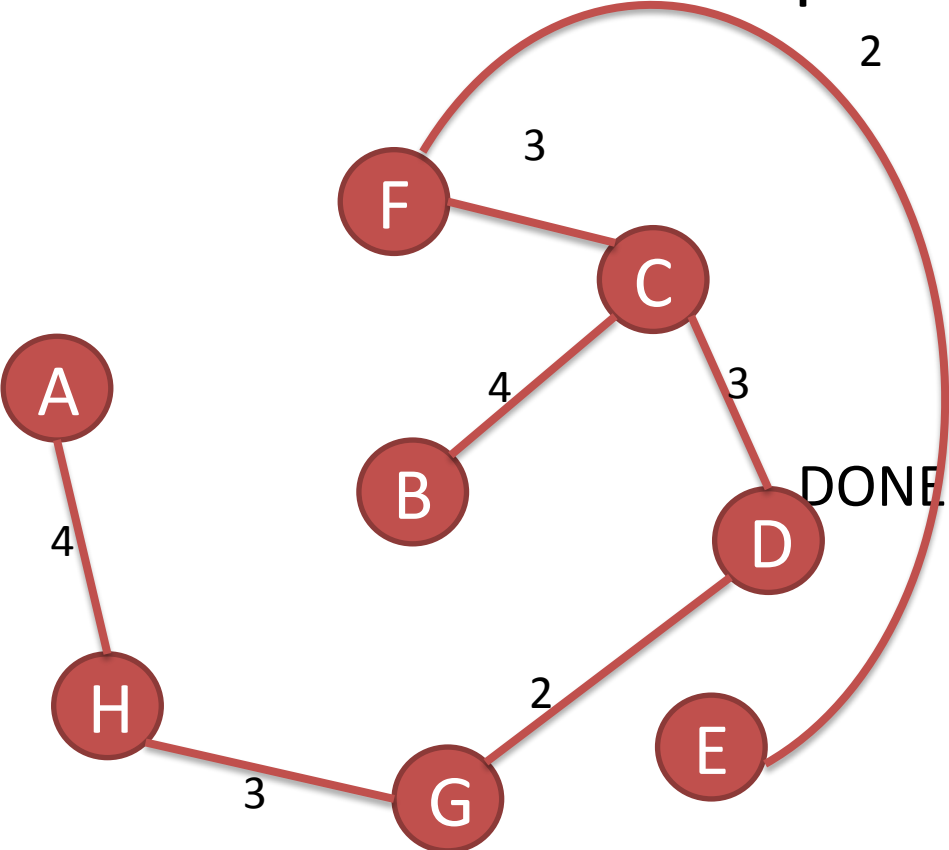
PRIM'S ALGORITHM

– k_v : has v been visited?

– d_v : What is the minimal edge weight to v ?

– p_v : What vertex precedes (is parent of) v ?

Cost of Minimum Spanning Tree =21



	K	d_v	P_v
A	T	4	H
B	T	4	C
C	T	3	D
D	T	0	--
E	T	2	F
F	T	3	C
G	T	2	D
H	T	3	G

Difference between Prim's and Kruskal's algorithm

Prims	Kruskal's
Prim's algorithm initializes with a node	Kruskal's algorithm initiates with an edge
Prim's algorithms span from one node to another	Kruskal's algorithm select the edges in a way that the position of the edge is not based on the last step
In prim's algorithm, graph must be a connected graph	Kruskal's can function on disconnected graphs too.

Applications

- Topology
- Geometry
- Clustering
- Routing Algorithms
- Generation of Mazes